Multilingual Tokenization Efficiency in Large Language Models: A Study on Indian Languages

Mohamed Azharudeen M, Balaji Dhamodharan
romoazhar007@gmail.com, Balaji.dhamodhar@gmail.com

I. ABSTRACT

This study evaluates several multilingual Large Language Models (LLMs), including popular models such as BLOOM, XGLM, Llama, Mistral and Gemma, to explore the efficiency of different tokenizers in processing Indian languages. Our analysis focuses on the tokenization process, which is critical for managing the diverse character sets of languages like Assamese, Bengali, English, Gujarati, Hindi, Kannada, Malayalam, Marathi, Nepali, Oriya, Punjabi, Tamil, Telugu and Urdu. Results indicate that the BLOOM tokenizer significantly outperformed others in terms of efficiency, processing 549,000 characters in just 5011.131471 seconds at a cost of $11.42538, marking it as the most cost-effective option. This comparative study provides insights into the token generation impacts on computational costs and processing time, offering a baseline for tokenizer performance across multiple languages. For detailed experimental results, readers are referred to the results section or to the findings section (XI).

II. INTRODUCTION

Large language models (LLMs) have emerged as significant milestones in the field of natural language processing, showcasing extraordinary capabilities across a variety of downstream tasks including summarization, reading comprehension, translation, and commonsense reasoning ([7]Brown et al., 2020b; [8]Touvron et al., 2023). At the heart of training these powerful models lies the tokenizer, a critical component that segments training documents into tokens—be the words, sub-words, or characters ([6]Bengio et al., 2000; [9]Schuster and Nakajima, 2012; [10]Sennrich et al., 2015; [11]Wang et al., 2020). These tokens, each represented by an embedding vector within the model, form the groundwork for all subsequent processing and learning phases.

Our research is prompted by the recognition of a significant gap in current LLM research—the nuanced role of tokenizers, especially in the context of Indian languages. These languages, representing a complex linguistic mosaic, are often marginalized in the shadow of dominant English-centric models. Considering that English is the primary language for just about 20% of the world's population, our study becomes crucial to ensure the inclusivity and representativeness of the global majority. Languages such as Tamil, Telugu, Malayalam, and Hindi exhibit distinct linguistic features that deserve recognition and adequate representation in LLMs.

This study rigorously examines the efficiency of various state-of-the-art pretrained models' tokenizers in processing the rich character sets unique to Indian languages. Our goal is to decipher the subtleties of token generation and assess its overarching impact on the multilingual performance of LLMs, with the intention of identifying the most efficient tokenizer for Indian languages. Employing a comparative approach, we evaluate token generation across different languages using a consistent text length, hypothesizing that fewer tokens correlate with heightened model efficiency.

Additionally, our research delves into a cost-benefit analysis through the use of heat maps, providing a visual representation of the economic considerations inherent in tokenizer selection. This analysis confirms the necessity for strategic model selection, the most cost-effective tokenizer, requiring considerably less expenditure than its counterparts. These observations highlight a tokenizer efficiency continuum, advocating that a judicious choice of tokenizer can result in marked cost savings and enhanced operational efficiency—key factors for stakeholders in the natural language processing industry.

III. RELATED WORK

This section synthesizes existing research on tokenization methodologies and their integration within transformer-based language models, both encoder-only and decoder-only architectures.

A. Tokenization Approaches

1) Word Tokenization: The fundamental tokenization method involves segmenting text based on spaces, treating each discrete word as a token. This was an early approach detailed by [6]Bengio et al. (2000), which, while straightforward, is limited by its inability to handle out-of-vocabulary (OOV) words effectively.

2) Subword Tokenization: The evolution of tokenization has led to data-driven techniques capable of deconstructing words into subword units. These methods have become the cornerstone upon which contemporary LLMs are constructed, enabling the processing of OOV words through the combination of subword tokens in the existing vocabulary ([12]Kudo and Richardson, 2018). Notable algorithms include WordPiece ([9]Schuster and Nakajima, 2012), BPE ([13]Gage, 1994; [10]Sennrich et al., 2015), Byte-Level BPE (BBPE) ([11]Wang et al., 2020), and Unigram ([?]Kudo, 2018). Recent advancements by [15]Petrov et al. (2023) have further refined these approaches.

3) Character Tokenization: Tokenization at the character level, or using UTF-8 bytes, is another strategy that, despite increasing sequence length and subsequent computational demands due to the quadratic complexity in transformer models, remains relevant ([16]Vaswani et al., 2017). Innovations to mitigate computational costs associated with character tokenization have been proposed ([17]Gao et al., 2020b; [18]Tay et al., 2021; [19]Xue et al., 2022; [20]Clark et al., 2022; [21]Yu et al., 2023).

B. Tokenizers in Transformer Models

1) Tokenizers in Encoder Models: This research predominantly targets encoder models, examining the impact of tokenization on downstream performance. [22]Rust et al. (2021) explored this within the BERT framework, while [23]Zhang et al. (2022) correlated tokenizer training with machine translation efficacy. The work of [24]Toraman et al. (2023) compares subword tokenizers in Turkish medium-sized language models, finding negligible performance differences compared to more pronounced disparities with word and character-level tokenizers. [25]Chirkova and Troshin (2022) highlighted the influence of different tokenizations on code-related tasks, underscoring the potential of optimized tokenizers to significantly reduce sequence lengths or marginally enhance performance.

2) Tokenizers in Decoder Models: A comprehensive review of mono- and multilingual LLMs is offered by [26]Lin et al. (2022), [27]Shliazhko et al. (2022), and [28]Scao et al. (2022). [29]Stollenwerk (2023) delved into the intrinsic properties of the GPT-SW3 tokenizer, with a focus on Nordic languages. Additionally, [27]Shliazhko et al. (2022) scrutinized various tokenizer pre-processing strategies to identify their impact on model performance. However, extensive analysis of extrinsic tokenizer performance remains a less explored area in major LLM research.

IV. Our Approach

A. Objective

The foundational objective of this research is to rigorously assess the tokenization efficiency of multilingual large language models (LLMs), with a concentrated focus on a diverse set of Indian languages. This objective stems from the understanding that the efficiency of tokenization directly influences the overall performance and cost, especially in dealing with languages that have varying linguistic structures and complexities.

B. Selection of Language Models

Our research conducts a comprehensive analysis of 106 popular, open-source pretrained LLMs. These models vary in complexity, with parameters ranging from 2 billion to 175 billion. The selection includes diverse models like Qwen/Qwen1.5-72B, chargoddard/Yi-34B-Llama, internlm/internlm2-20b, among others, providing a robust framework for comparison and evaluation.

C. Language Selection

The study focuses on a fixed ensemble of Indian languages, each representing diverse linguistic families and scripts. These languages encompass Tamil, English, Urdu, Hindi, Nepali, Marathi, Punjabi, Oriya, Gujarati, Assamese, Bengali, Telugu, Kannada, and Malayalam. This choice is driven by the intent to cover a broad spectrum of linguistic attributes, contributing to a comprehensive understanding of tokenization across varied language systems.

D. Standardized Character Lengths

To ensure consistent and uniform analysis, texts in each selected language have been standardized to specific character lengths: 4, 8, 16, 32, 64, 128, 256, and 512. This standardization is crucial in establishing a baseline for accurately and comparatively evaluating tokenization efficiency across various models and languages. Furthermore, to determine token size for a larger number of characters, the forecasting values derived from this set of character lengths provide approximately 95% accuracy with a tolerance rate of 5%.

E. Tokenization Process Analysis

At the heart of this study is the in-depth analysis of the tokenization process. This includes quantifying the number of tokens generated for each pre-determined text length and systematically comparing the token counts across various models. Efficiency is evaluated based on the number of tokens generated, with a lower token count for equivalent information indicating higher efficiency.

V. Experimental Setup

A. Forecasting Approach for Token Prediction

In anticipation of real-world applications, such as processing extensive texts like books, we employ a forecasting approach for token prediction. This approach uses the average number of characters in a book, estimated to be around 549,000 including spaces and new lines (reference IX), as a realistic benchmark for extrapolation.

B. Mathematical Model

The research employs an exponential growth model for forecasting token values, underpinned by the assumption that character counts (steps) double at each successive interval. The model is mathematically represented as:

$$y = a \cdot e^{b \cdot x} \tag{1}$$

Here,

- y symbolizes the predicted token value,
- x is the natural logarithm of the step count (character count), and
- a and b are coefficients to be ascertained through the model fitting process using the token values for standardized character lengths.

C. Coefficient Determination

The coefficients a and b will be derived via a rigorous curve-fitting process. This process entails the transformation of character lengths through natural logarithms, facilitating an effective fitting of the exponential growth model to the data gathered from various LLMs across all selected languages.

D. Extrapolating Token Values

The model will then be employed to extrapolate token values for a character length of 549,000. This step involves calculating the natural logarithm of 549,000 and using the fitted model to predict the corresponding token count.

VI. Results

A. Comparative Analysis

Our study meticulously evaluated tokenization efficiency across a range of large language model (LLM) tokenizers. We analyzed the performance of these tokenizers by comparing token generation across a corpus comprising 14 languages, represented at various character lengths. The tokenizers listed below represent the top five best and worst performers. The top five in the list are the best-performing tokenizers, while the bottom five are the worst-performing tokenizers, evaluated across all languages and different character sizes of token generation. For a comprehensive overview of overall tokenizer performance, please refer to the findings section (XI). For individual language-level performance, refer Tables II to XV.

- bigscience/bloom
- bigscience/bloom-7b1
- croissantllm/CroissantLLMBase
- facebook/xglm-7.5B
- abhinand/gemma-2b-tamil
- internlm/internlm2-20b
- nxn1231/yi6
- liminerity/mma-4-3b
- intervirens/internlm2-base-20b-llama
- yanolja/EEVE-Korean-2.8B-v1.0

The analysis highlighted that models such as BigScience's Bloom, CroissantLLMBase, and Facebook's xglm-7.5B, along with the Gemma-based models, achieved commendable efficiency. These models stood out not only in overall performance but also in adeptly handling the nuances of specific languages.

1) Token Size Differentiation: A significant outcome of this research is the discovery of a considerable disparity in the number of tokens generated by the least and most efficient tokenizers. This discrepancy reveals substantial variability in the performance capacities of these tokenizers. For example, the tokenizer yanolja/EEVE-Korean-2.8B-v1.0 generated an average of 1,273,169 tokens for the standard 549,000 characters across all evaluated languages. In stark contrast, the best-performing model, bigscience/bloom, required approximately 95,211 tokens for the same character count. The differential token count between these models is striking, standing at around 1,177,957 tokens. This denotes that selecting the appropriate tokenizer for a given language could result in a substantial economy of tokens, in this case, saving approximately 1.18 million tokens.

2) Detailed Language-Specific Token Analysis: An intricate examination at the individual language level for all 14 languages included in this study revealed:

- **BigScience's Bloom Model:** Consistently performed well across the board, showing remarkable efficiency in tokenizing complex texts.
- **Least Performing Tokenizer:** The yanolja/EEVE-Korean-2.8B-v1.0 model exhibited the least efficiency, resulting in the highest token counts for the corpus used.
- **Token Generation Spectrum:** The range of tokens generated spanned from just under a hundred thousand to over a million, underscoring the critical importance of model selection in relation to language-specific tokenization requirements.
- **Model Selection:** The findings suggest that optimal model selection could lead to not only improved computational efficiency but also cost savings, especially when processing large volumes of text data.

The Fig: 1 provided illustrates the token count for different character sizes across models, indicating a clear trend that some LLMs are more adept at handling longer character sequences efficiently. It is evident that certain models manage to maintain a lower token count even as the complexity of the text (character length) increases, which is indicative of their advanced tokenization capabilities.

Important to note: In the charts, after the character counts of 256 and 512, there is a jump to 92010. This was done for two main reasons: firstly, if we continued the axis with increments such as 512, 1024, 2048, and so on, the chart would become excessively long due to our target character count of 549,000. Secondly, we observed minimal differences in cost between these incremental steps. Therefore, after the 512 mark, we chose to analyze five representative numbers between 512 and 549,000 to conduct our study more efficiently.

Note: For the individual language-based analysis charts, please refer to the figures located after the References section.

B. Cost-Effectiveness Identification

Our research presents a comprehensive cost analysis comparing the tokenization efficiency of various large language model (LLM) tokenizers for Indian languages. Utilizing a heat map (Fig: 2) for an intuitive understanding of our findings, we examined the cost implications of tokenizer performance across several character lengths.

1) Tokenization Cost Variance: The tokenizers listed below represent the top five best and worst performers. The top five in the list are the best-performing tokenizers, while the bottom five are the worst-performing tokenizers, evaluated across all languages and various character sizes in terms of token generation and cost consumption. For a comprehensive overview of overall tokenizer performance, please refer to the findings section (XI). For individual language-level performance, refer Tables II to XV.

- bigscience/bloom
- bigscience/bloom-7b1
- croissantllm/CroissantLLMBase
- facebook/xglm-7.5B
- abhinand/gemma-2b-tamil
- internlm/internlm2-20b
- nxn1231/yi6
- liminerity/mma-4-3b
- intervirens/internlm2-base-20b-llama
- yanolja/EEVE-Korean-2.8B-v1.0

Our heat map visualization, as illustrated in Figure 2, underscores the vast cost disparities across different tokenizers. When evaluating efficiency, the tokenizer 'bigscience/bloom' stood out as the most cost-effective option, necessitating only $11.42538 to process the benchmark of 549,000 characters. Conversely, the tokenizer 'yanolja/EEVE-Korean-2.8B-v1.0' presented a considerably higher expense, amounting to $152.780268 for processing the identical character count. This estimation assumes a price of $0.00012 per token refer: X, as specified by OpenAI for the GPT-4-32k model at the current rate (for more details on pricing, visit OpenAI's Pricing Page). The costs are derived by multiplying the number of tokens required by the tokenizer to process the character count with the cost per token, leveraging the pricing formula:

$$\text{Total Cost} = \text{Number of Tokens} \times \text{Cost Per Token} \tag{2}$$

ensuring a precise and transparent financial assessment.

Important to note: In the charts, after the character counts of 256 and 512, there is a jump to 92010. This was done for two main reasons: firstly, if we continued the axis with increments such as 512, 1024, 2048, and so on, the chart would become excessively long due to our target character count of 549,000. Secondly, we observed minimal differences in cost between these incremental steps. Therefore, after the 512 mark, we chose to analyze five representative numbers between 512 and 549,000 to conduct our study more efficiently.

2) Cost Implications of Tokenizer Selection: The stark contrast in tokenization costs between the best and worst-performing models, amounting to $141.3, emphasizes the substantial financial impact of tokenizer choice. Such a cost differential makes a compelling case for careful model selection, particularly for applications involving the processing of large volumes of text in multilingual contexts.

3) Language-Specific Cost Analysis: An in-depth analysis of tokenization costs at the individual language level yielded the following insights:

- **Best Performing Tokenizer:** bigscience/bloom stood out for its consistent performance across different Indian languages, demonstrating a high degree of efficiency in tokenizing complex scripts.
- **Least Efficient Tokenizer:** The model from yanolja/EEVE-Korean-2.8B-v1.0, was found to be the least cost-efficient, reflecting the necessity for model optimization tailored to the intricacies of Indian languages.
- **Token Generation Cost Spectrum:** There was a broad spectrum of token generation costs, highlighting the importance of selecting a model that is not only linguistically competent but also cost-efficient.
- **Model Selection for Cost Savings:** Optimal tokenizer selection has proven computational efficiency and substantial cost savings, a critical consideration for stakeholders in the field of natural language processing.

The accompanying heat map (Fig: 2) distinctly shows the token generation costs for different character sizes, illustrating that certain models exhibit proficiency in maintaining lower token counts even as character complexity increases. This reveals their superior tokenization capabilities, directly translating into cost efficiency for end-users.
Note: For the individual language-based analysis charts, please refer to the figures located after the References section.

C. Token Generation Time Analysis

Our research extends beyond simple tokenization efficiency to examine the time required to process extensive character sequences across various large language model (LLM) tokenizers. This metric is crucial for real-world applications where processing time is directly correlated with operational efficiency and cost.

The tokenizers listed below represent the top five best and worst performers. The top five in the list are the best-performing tokenizers, while the bottom five are the worst-performing tokenizers, evaluated across all languages and various character sizes in terms of token generation and time consumption. For a comprehensive overview of overall tokenizer performance, please refer to the findings section (XI). For individual language-level performance, refer Tables II to XV.

- bigscience/bloom
- bigscience/bloom-7b1
- croissantllm/CroissantLLMBase
- facebook/xglm-7.5B
- abhinand/gemma-2b-tamil
- internlm/internlm2-20b
- nxn1231/yi6
- liminerity/mma-4-3b
- intervirens/internlm2-base-20b-llama
- yanolja/EEVE-Korean-2.8B-v1.0

1) Throughput Impact on Tokenization: Utilizing the data visualized in the Fig: 3 bar plot, we assessed the time taken by each tokenizer to process for 549,000 characters. Our analysis revealed significant differences in processing times, where the most efficient model, bigscience/bloom, completed the task in just 5011.131471 seconds, while the least efficient, yanolja/EEVE-Korean-2.8B-v1.0, took a prolonged 67008.889377 seconds. This vast disparity of 61997.757906 seconds underscores the performance divergence across tokenizers.

2) Token Generation Time Impact on Tokenization: The variability in token generation times indicates a significant divergence in tokenizer performance, with some models capable of handling large character sequences more rapidly than others. This speed of processing not only impacts the timeliness of data output but also affects the economic aspects of operations in scenarios where computational resources are costed by runtime.

Our analysis demonstrates the importance of selecting a tokenizer that not only offers linguistic accuracy but also high operational speed. The ability to swiftly and effectively process vast text corpora is a pivotal factor for entities relying on LLMs for extensive multilingual text analysis and applications. Therefore, the efficiency of token generation becomes a key performance indicator for the adoption of LLMs in environments where time efficiency and cost-effectiveness are critical.
Note: For the individual language-based analysis charts, please refer to the figures located after the References section.

VII. Discussion

This study has systematically investigated the impact of tokenization efficiency on the utility and cost-effectiveness of multilingual models, particularly within the rich tapestry of India's linguistic diversity. Our findings shed light on how

different tokenization strategies can significantly affect both the performance and the economic viability of language models in handling diverse linguistic inputs.

A. Tokenizer Performance in Training and Inference

While the focus of this study was primarily on the inference and token generation aspects of language models, the impact of tokenization extends into the training phase as well. Effective tokenization can influence both the forward and backward passes during training, affecting everything from learning dynamics to model convergence rates. Our findings suggest a potential area for future research: exploring how different tokenization strategies impact the training efficiency and effectiveness of multilingual models.

B. Performance of Models with Varying Vocabulary Sizes

An intriguing aspect of our analysis was the performance comparison between models with large vocabularies, such as Gemma and Bloom, and those with smaller vocabularies, like CroissantLLM and xGLM. Despite the smaller vocabulary size, CroissantLLM and xGLM performed exceptionally well, challenging the conventional belief that larger vocabularies always yield better performance. This suggests that other factors, such as tokenizer architecture and optimization strategies, may play significant roles in determining the effectiveness of a model. Understanding the mechanisms behind the success of models with smaller vocabularies could provide valuable insights into the design of more efficient and adaptable tokenizers.

C. Future Research Directions

Given the comprehensive nature of this study, several avenues remain open for further exploration. Future research should delve deeper into the training aspects of tokenization, examining how different tokenizers affect model training dynamics and learning outcomes. Additionally, further investigation is needed to unravel why models like CroissantLLM and xGLM perform well despite their limited vocabulary sizes. Such studies could lead to breakthroughs in tokenizer design, potentially revolutionizing how language models are developed for multilingual applications.

In summary, our research underscores the critical importance of tokenizer efficiency in the development of multilingual language models. By fostering a deeper understanding of how tokenization impacts both the cost and effectiveness of language processing, this work contributes to the broader goal of creating more inclusive and accessible technology solutions for diverse linguistic communities.

VIII. Conclusion

Our study significantly advances the understanding of tokenizer efficiency within large language models (LLMs), particularly in multilingual contexts focusing on Indian languages. Our findings unveiled significant efficiencies and disparities among various tokenizers. Models like BigScience's Bloom, Facebook's xglm-7.5B, and CroissantLLM's CroissantLLMBase demonstrated remarkable capabilities in handling complex Indian language scripts, while others, notably Yanolja's EEVE-Korean-2.8B-v1.0, lagged in efficiency. This underscores the crucial role of model selection for linguistic inclusivity and computational practicality. Our thorough cost-benefit analysis, augmented by intuitive heat map visualizations, revealed marked cost differences among tokenizers, emphasizing the necessity for strategic model selection in language processing applications. The research highlights how tokenizer choice profoundly impacts performance, cost, and operational efficiency, making a strong case for the focused development of tokenizers that adeptly manage linguistic diversity. This study charts a path towards more inclusive, efficient, and economically viable language models in the realm of natural language processing.

[Note: All authors contributed equally and fairly]

ACKNOWLEDGMENT

We express our sincere gratitude to Hugging Face for providing the resources crucial for our analysis of tokenizer efficiency. Their extensive toolkit has greatly enhanced our research capabilities, helping us push the boundaries of natural language processing. We also thank the developers of the individual models and tokenizers used in our study for their foundational contributions to our research. Their efforts in developing these open-source tools have been indispensable. We regret any unintentional omissions of specific contributors from our references and appreciate the opportunity to utilize such significant resources.

REFERENCES

[1] Abhinand Balachandran, Tamil-Llama: A New Tamil Language Model Based on Llama 2 https://arxiv.org/abs/2311.05845v1. [Accessed: 2023]

[2] Mehdi Ali et al., Tokenizer Choice For LLM Training: Negligible or Crucial https://arxiv.org/abs/2310.08754v4. [Accessed: 2023].

[3] Mitodru Niyogi, Arnab Bhattacharya, Paramanu: A Family of Novel Efficient Indic Generative Foundation Language Models https://arxiv.org/abs/2401.18034v1. [Accessed: 2024].

[4] Wei Qi Leong et al., BHASA: A Holistic Southeast Asian Linguistic and Cultural Evaluation Suite for Large Language Models https://arxiv.org/abs/2309.06085v2. [Accessed: 2023].

[5] Chenxi Whitehouse, Monojit Choudhury, Alham Fikri Aji, LLM-powered Data Augmentation for Enhanced Cross-lingual Performance. https://arxiv.org/abs/2305.14288v2. [Accessed: 2023].

[6] Yoshua Bengio, Réjean Ducharme, and Pascal Vincent. 2000. A neural probabilistic language model. Advances in neural information processing systems, 13.

[7] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020a. Language models are few-shot learners. Advances in neural information processing systems, 33:1877–1901.

[8] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: open and efficient foundation language models, 2023. URL https://arxiv.org/abs/2302.13971.

[9] Mike Schuster and Kaisuke Nakajima. 2012. Japanese and korean voice search. In 2012 IEEE international conference on acoustics, speech and signal processing (ICASSP), pages 5149–5152. IEEE.

[10] Rico Sennrich, Barry Haddow, and Alexandra Birch. 2015. Neural machine translation of rare words with subword units. arXiv preprint arXiv:1508.07909.

[11] Changhan Wang, Kyunghyun Cho, and Jiatao Gu. 2020. Neural machine translation with byte-level subwords. Proceedings of the AAAI Conference on Artificial Intelligence, 34(05):9154–9160.

[12] Taku Kudo and John Richardson. 2018. Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. EMNLP 2018, page 66.

[13] Philip Gage. 1994. A new algorithm for data compression. The C Users Journal archive, 12:23–38.

[14] Taku Kudo. 2018. Subword regularization: Improving neural network translation models with multiple subword candidates. In Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 66–75, Melbourne, Australia. Association for Computational Linguistics

[15] Aleksandar Petrov, Emanuele La Malfa, Philip HS Torr, and Adel Bibi. 2023. Language model tokenizers introduce unfairness between languages. arXiv preprint arXiv:2305.15425.

[16] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In NIPS, pages 5998–6008.

[17] Yingqiang Gao, Nikola I. Nikolov, Yuhuang Hu, and Richard H.R. Hahnloser. 2020b. Character-level translation with self-attention. In Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, pages 1591–1604, Online. Association for Computational Linguistics.

[18] Yi Tay, Vinh Q Tran, Sebastian Ruder, Jai Gupta, Hyung Won Chung, Dara Bahri, Zhen Qin, Simon Baumgartner, Cong Yu, and Donald Metzler. 2021. Charformer: Fast character transformers via gradientbased subword tokenization. In International Conference on Learning Representations.

[19] Linting Xue, Aditya Barua, Noah Constant, Rami AlRfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. 2022. Byt5: Towards a token-free future with pre-trained byte-to-byte models. Transactions of the Association for Computational Linguistics, 10:291–306.

[20] Jonathan H. Clark, Dan Garrette, Iulia Turc, and John Wieting. 2022. Canine: Pre-training an efficient tokenization-free encoder for language representation. Transactions of the Association for Computational Linguistics, 10:73–91.

[21] Lili Yu, Dániel Simig, Colin Flaherty, Armen Aghajanyan, Luke Zettlemoyer, and Mike Lewis. 2023. Megabyte: Predicting million-byte sequences with multiscale transformers. arXiv preprint arXiv:2305.07185.

[22] Phillip Rust, Jonas Pfeiffer, Ivan Vulic, Sebastian Ruder, and Iryna Gurevych. 2021. How good is your tokenizer? on the monolingual performance of multilingual language models. In Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers), pages 3118–3135, Online. Association for Computational Linguistics.

[23] Shiyue Zhang, Vishrav Chaudhary, Naman Goyal, James Cross, Guillaume Wenzek, Mohit Bansal, and Francisco Guzman. 2022. How robust is neural machine translation to language imbalance in multilingual tokenizer training? In Proceedings of the 15th biennial conference of the Association for Machine Translation in the Americas (Volume 1: Research Track), pages 97–116, Orlando, USA. Association for Machine Translation in the Americas.

[24] Cagri Toraman, Eyup Halit Yilmaz, Furkan Sahinuc, and Oguzhan Ozcelik. 2023. Impact of tokenization on language models: An analysis for turkish. ACM Trans. Asian Low-Resour. Lang. Inf. Process.

[25] Nadezhda Chirkova and Sergey Troshin. 2022. CodeBPE: Investigating subtokenization options for large language model pretraining on source code. In Deep Learning for Code Workshop.

[26] Xi Victoria Lin, Todor Mihaylov, Mikel Artetxe, Tianlu Wang, Shuohui Chen, Daniel Simig, Myle Ott, Naman Goyal, Shruti Bhosale, Jingfei Du, et al. 2022. Few-shot learning with multilingual generative language models. In Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, pages 9019–9052.

[27] Oleh Shliazhko, Alena Fenogenova, Maria Tikhonova, Vladislav Mikhailov, Anastasia Kozlova, and Tatiana Shavrina. 2022. mgpt: Few-shot learners go multilingual. arXiv preprint arXiv:2204.07580.

[28] Teven Le Scao, Angela Fan, Christopher Akiki, Ellie Pavlick, Suzana Ilic, Daniel Hesslow, Roman Castagné, Alexandra Sasha Luccioni, François Yvon, Matthias Gallé, Jonathan Tow, Alexander M. Rush, Stella Biderman, Albert Webson, Pawan Sasanka Ammanamanchi, Thomas Wang, Benoît Sagot, Niklas Muennighoff, Albert Villanova del Moral, Olatunji Ruwase, Rachel Bawden, Stas Bekman, Angelina McMillan-Major, Iz Beltagy, Huu Nguyen, Lucile Saulnier, Samson Tan, Pedro Ortiz Suarez, Victor Sanh, Hugo Laurençon, Yacine Jernite, Julien Launay, Margaret Mitchell, Colin Raffel, Aaron Gokaslan, Adi Simhi, Aitor Soroa, Alham Fikri Aji, Amit Alfassy, Anna Rogers, Ariel Kreisberg Nitzav, Canwen Xu, Chenghao Mou, Chris Emezue, Christopher Klamm, Colin Leong, Daniel van Strien, David Ifeoluwa Adelani, and et al. 2022. BLOOM: A 176b-parameter open-access multilingual language model. CoRR, abs/2211.05100.

[29] Felix Stollenwerk. 2023. Training and evaluation of a multilingual tokenizer for gpt-sw3.

[30] Shekhar et al, Towards Optimizing the Costs of LLM Usage https://arxiv.org/abs/2402.01742v1. [Accessed: 2024].

[31] Ninglu et al, Flexibly Scaling Large Language Models Contexts Through Extensible Tokenization https://arxiv.org/abs/2401.07793v1. [Accessed: 2024].

[32] Gemma Team et al, Gemma: Open Models Based on Gemini Research and Technology https://arxiv.org/abs/2403.08295. [Accessed: 2024].

[33] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, William El Sayed, Mistral 7B https://arxiv.org/abs/2310.06825. [Accessed: 2023].

[34] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, Guillaume Lample, LLaMA: Open and Efficient Foundation Language Models. https://arxiv.org/abs/2302.13971. [Accessed: 2023].

[35] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, Thomas Scialom, Llama 2: Open Foundation and Fine-Tuned Chat Models. https://arxiv.org/abs/2307.09288. [Accessed: 2023].

[36] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, Binyuan Hui, Luo Ji, Mei Li, Junyang Lin, Runji Lin, Dayiheng Liu, Gao Liu, Chengqiang Lu, Keming Lu, Jianxin Ma, Rui Men, Xingzhang Ren, Xuancheng Ren, Chuanqi Tan, Sinan Tan, Jianhong Tu, Peng Wang, Shijie Wang, Wei Wang, Shengguang Wu, Benfeng Xu, Jin Xu, An Yang, Hao Yang, Jian Yang, Shusheng Yang, Yang Yao, Bowen Yu, Hongyi Yuan, Zheng Yuan, Jianwei Zhang, Xingxuan Zhang, Yichang Zhang, Zhenru Zhang, Chang Zhou, Jingren Zhou, Xiaohuan Zhou, Tianhang Zhu. Qwen Technical Report. https://arxiv.org/abs/2309.16609. [Accessed: 2023].

[37] Manuel Faysse, Patrick Fernandes, Nuno M. Guerreiro, António Loison, Duarte M. Alves, Caio Corro, Nicolas Boizard, João Alves, Ricardo Rei, Pedro H. Martins, Antoni Bigata Casademunt, François Yvon, André F.T. Martins, Gautier Viaud, Céline Hudelot, Pierre Colombo. CroissantLLM: A Truly Bilingual French-English Language Model. https://arxiv.org/abs/2402.00786. [Accessed: 2024].

[38] Xi Victoria Lin, Todor Mihaylov, Mikel Artetxe, Tianlu Wang, Shuohui Chen, Daniel Simig, Myle Ott, Naman Goyal, Shruti Bhosale, Jingfei Du, Ramakanth Pasunuru, Sam Shleifer, Punit Singh Koura, Vishrav Chaudhary, Brian O'Horo, Jeff Wang, Luke Zettlemoyer, Zornitsa Kozareva, Mona Diab, Veselin Stoyanov, Xian Li. Few-shot Learning with Multilingual Language Models. https://arxiv.org/abs/2112.10668. [Accessed: 2022].

[39] InternLM. Internlm: A multilingual language model with progressively enhanced capabilities. https://github.com/InternLM/InternLM, [Accessed: 2023].

[40] Seungduk Kim, Seungtaek Choi1, Myeongho Jeong Yanolja. Efficient and Effective Vocabulary Expansion Towards Multilingual Large Language Models. https://arxiv.org/html/2402.14714v1. [Accessed: 2024].

[41] 01.AI. Yi: Open Foundation Models by 01.AI. https://arxiv.org/html/2403.04652v1. [Accessed: 2024].

[42] DeepSeek-AI: Xiao Bi, Deli Chen, Guanting Chen, Shanhuang Chen, Damai Dai, Chengqi Deng, Honghui Ding, Kai Dong, Qiushi Du, Zhe Fu, Huazuo Gao, Kaige Gao, Wenjun Gao, Ruiqi Ge, Kang Guan, Daya Guo, Jianzhong Guo, Guangbo Hao, Zhewen Hao, Ying He, Wenjie Hu, Panpan Huang, Erhang Li, Guowei Li, Jiashi Li, Yao Li, Y.K. Li, Wenfeng Liang, Fangyun Lin, A.X. Liu, Bo Liu, Wen Liu, Xiaodong Liu, Xin Liu, Yiyuan Liu, Haoyu Lu, Shanghao Lu, Fuli Luo, Shirong Ma, Xiaotao Nie, Tian Pei, Yishi Piao, Junjie Qiu, Hui Qu, Tongzheng Ren, Zehui Ren, Chong Ruan, Zhangli Sha, Zhihong Shao, Junxiao Song, Xuecheng Su, Jingxiang Sun, Yaofeng Sun, Minghui Tang, Bingxuan Wang, Peiyi Wang, Shiyu Wang, Yaohui Wang, Yongji Wang, Tong Wu, Y. Wu, Xin Xie, Zhenda Xie, Ziwei Xie, Yiliang Xiong, Hanwei Xu, R.X. Xu, Yanhong Xu, Dejian Yang, Yuxiang You, Shuiping Yu, Xingkai Yu, B. Zhang, Haowei Zhang, Lecong Zhang, Liyue Zhang, Mingchuan Zhang, Minghua Zhang, Wentao Zhang, Yichao Zhang, Chenggang Zhao, Yao Zhao, Shangyan Zhou, Shunfeng Zhou, Qihao Zhu, Yuheng Zou. DeepSeek LLM: Scaling Open-Source Language Models with Longtermism. https://arxiv.org/abs/2401.02954. [Accessed: 2024].

[43] Yiming Cui, Xin Yao. Rethinking LLM Language Adaptation: A Case Study on Chinese Mixtral https://arxiv.org/abs/2403.01851. [Accessed: 2024].

[44] Dahyun Kim, Chanjun Park, Sanghoon Kim, Wonsung Lee, Wonho Song Yunsu Kim, Hyeonwoo Kim, Yungi Kim, Hyeonju Lee, Jihoo Kim Changbae Ahn, Seonghoon Yang, Sukyung Lee, Hyunbyung Park, Gyoungjin Gim Mikyoung Cha, Hwalsuk Lee, Sunghun Kim. SOLAR 10.7B: Scaling Large Language Models with Simple yet Effective Depth Up-Scaling https://arxiv.org/html/2312.15166v2. [Accessed: 2023].

[45] Yuanzhi Li, Sébastien Bubeck, Ronen Eldan, Allie Del Giorno, Suriya Gunasekar, Yin Tat Lee. Textbooks Are All You Need II: phi-1.5 technical report https://arxiv.org/abs/2309.05463. [Accessed: 2023].

[46] Replete-AI/Mistral-Evolved-11b-v0.1. https://huggingface.co/Replete-AI/Mistral-Evolved-11b-v0.1, [Accessed: 2024].

[47] SGEcon/EconFinKoSOLAR-10.7B-SFT. https://huggingface.co/SGEcon/EconFinKoSOLAR-10.7B-SFT, [Accessed: 2024].

[48] Ruyi Gan, Ziwei Wu, Renliang Sun, Junyu Lu, Xiaojun Wu, Dixiang Zhang, Kunhao Pan, Junqing He, Yuanhe Tian, Ping Yang, Qi Yang, Hao Wang, Jiaxing Zhang, Yan Song. Ziya2: Data-centric Learning is All LLMs Need. https://arxiv.org/abs/2311.03301, [Accessed: 2024].

[49] amu/spin-phi2. https://huggingface.co/amu/spin-phi2, [Accessed: 2024].

[50] DeciLM-7B: Pushing the Boundaries of Text Generation Efficiency & Accuracy. https://huggingface.co/Deci/DeciLM-7B, [Accessed: 2023].

[51] Marah Abdin, Jyoti Aneja, Sebastien Bubeck, Caio César Teodoro Mendes, Weizhu Chen, Allie Del Giorno, Ronen Eldan, Sivakanth Gopi, Suriya Gunasekar, Mojan Javaheripi, Piero Kauffmann, Yin Tat Lee, Yuanzhi Li, Anh Nguyen, Gustavo de Rosa, Olli Saarikivi, Adil Salim, Shital Shah, Michael Santacroce, Harkirat Singh Behl, Adam Taumann Kalai, Xin Wang, Rachel Ward, Philipp Witte, Cyril Zhang, Yi Zhang. Phi-2: The surprising power of small language models https://www.microsoft.com/en-us/research/blog/phi-2-the-surprising-power-of-small-language-models/, [Accessed: 2023].

[52] TencentARC/Mistral-Pro-8B-v0.1. https://huggingface.co/TencentARC/Mistral-Pro-8B-v0.1, [Accessed: 2024].

[53] andysalerno/mistral-sft-v3. https://huggingface.co/andysalerno/mistral-sft-v3, [Accessed: 2024].

[54] awnr/Mistral-7B-v0.1-half-naive-A. https://huggingface.co/awnr/Mistral-7B-v0.1-half-naive-A, [Accessed: 2024].

[55] unsloth/mistral-7b-v0.2. https://huggingface.co/unsloth/mistral-7b-v0.2, [Accessed: 2024].

[56] codellama/CodeLlama-70b-Instruct-hf. https://huggingface.co/codellama/CodeLlama-70b-Instruct-hf, [Accessed: 2024].

[57] codellama/CodeLlama-70b-hf. https://huggingface.co/codellama/CodeLlama-70b-hf, [Accessed: 2024].

[58] Zhikun Xu, Yinghui Li, Ruixue Ding, Xinyu Wang, Boli Chen, Yong Jiang, Hai-Tao Zheng, Wenlian Lu, Pengjun Xie, Fei Huang. Let LLMs Take on the Latest Challenges! A Chinese Dynamic Question Answering Benchmark. https://arxiv.org/abs/2402.19248. [Accessed: 2024].

[59] Sailor: Open Language Models for South-East Asia. https://github.com/sail-sg/sailor-llm. [Accessed: 2024].

[60] liminerity/mm4-3b. https://huggingface.co/liminerity/mm4-3b. [Accessed: 2024].

[61] nnethercott/llava-v1.5-7b-hf-vicuna. https://huggingface.co/nnethercott/llava-v1.5-7b-hf-vicuna. [Accessed: 2024].

[62] Jun Liu, Chao Wu, Changdi Yang, Hao Tang, Haoye Dong, Zhenglun Kong, Geng Yuan, Wei Niu, Dong Huang, Yanzhi Wang. Efficient Pruning of Large Language Model with Adaptive Estimation Fusion https://arxiv.org/html/2403.10799v1. [Accessed: 2024].

[63] itsliupeng/openllama-7b-icl. https://huggingface.co/itsliupeng/openllama-7b-icl. [Accessed: 2024].

[64] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, Todor Mihaylov, Myle Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Singh Koura, Anjali Sridhar, Tianlu Wang, Luke Zettlemoyer. OPT: Open Pre-trained Transformer Language Models https://arxiv.org/abs/2205.01068. [Accessed: 2022].

[65] Sid Black, Stella Biderman, Eric Hallahan, Quentin Anthony, Leo Gao, Laurence Golding, Horace He, Connor Leahy, Kyle McDonell, Jason Phang, Michael Pieler, USVSN Sai Prashanth, Shivanshu Purohit, Laria Reynolds, Jonathan Tow, Ben Wang, Samuel Weinbach. GPT-NeoX-20B: An Open-Source Autoregressive Language Model. https://arxiv.org/abs/2204.06745. [Accessed: 2022].

[66] togethercomputer/RedPajama-INCITE-7B-Base.https://huggingface.co/togethercomputer/RedPajama-INCITE-7B-Base. [Accessed: 2023].
[67] Zhengzhong Liu, Aurick Qiao, Willie Neiswanger, Hongyi Wang, Bowen Tan, Tianhua Tao, Junbo Li, Yuqi Wang, Suqi Sun, Omkar Pangarkar, Richard Fan, Yi Gu, Victor Miller, Yonghao Zhuang, Guowei He, Haonan Li, Fajri Koto, Liping Tang, Nikhil Ranjan, Zhiqiang Shen, Xuguang Ren, Roberto Iriondo, Cun Mu, Zhiting Hu, Mark Schulze, Preslav Nakov, Tim Baldwin, Eric P. Xing. LLM360: Towards Fully Transparent Open-Source LLMs. https://arxiv.org/abs/2312.06550. [Accessed: 2023].
[68] NucleusAI/nucleus-22B-token-500B. https://huggingface.co/NucleusAI/nucleus-22B-token-500B. [Accessed: 2023].
[69] EleutherAI/gpt-j-6b. https://huggingface.co/EleutherAI/gpt-j-6b. [Accessed: 2023].
[70] Stella Biderman, Hailey Schoelkopf, Quentin Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, Oskar van der Wal. Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling. https://arxiv.org/abs/2304.01373. [Accessed: 2023].
[71] Philipp Singer, Pascal Pfeiffer, Yauhen Babakhin, Maximilian Jeblick, Nischay Dhankhar, Gabor Fodor, Sri Satish Ambati. H2O-Danube-1.8B Technical Report. https://arxiv.org/abs/2401.16818. [Accessed: 2024].
[72] Nolan Dey, Gurpreet Gosal, Zhiming (Charles)Chen, Hemant Khachane, William Marshall, Ribhu Pathria, Marvin Tom, Joel Hestness. Cerebras-GPT: Open Compute-Optimal Language Models Trained on the Cerebras Wafer-Scale Cluster. https://arxiv.org/abs/2304.03208. [Accessed: 2023].
[73] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, Qian Liu, Evgenii Zheltonozhskii, Terry Yue Zhuo, Thomas Wang, Olivier Dehaene, Mishig Davaadorj, Joel Lamy-Poirier, João Monteiro, Oleh Shliazhko, Nicolas Gontier, Nicholas Meade, Armel Zebaze, Ming-Ho Yee, Logesh Kumar Umapathi, Jian Zhu, Benjamin Lipkin, Muhtasham Oblokulov, Zhiruo Wang, Rudra Murthy, Jason Stillerman, Siva Sankalp Patel, Dmitry Abulkhanov, Marco Zocca, Manan Dey, Zhihan Zhang, Nour Fahmy, Urvashi Bhattacharyya, Wenhao Yu, Swayam Singh, Sasha Luccioni, Paulo Villegas, Maxim Kunakov, Fedor Zhdanov, Manuel Romero, Tony Lee, Nadav Timor, Jennifer Ding, Claire Schlesinger, Hailey Schoelkopf, Jan Ebert, Tri Dao, Mayank Mishra, Alex Gu, Jennifer Robinson, Carolyn Jane Anderson, Brendan Dolan-Gavitt, Danish Contractor, Siva Reddy, Daniel Fried, Dzmitry Bahdanau, Yacine Jernite, Carlos Muñoz Ferrandis, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro von Werra, Harm de Vries. StarCoder: may the source be with you!. https://arxiv.org/abs/2305.06161. [Accessed: 2023].
[74] Hyunwoong Ko, Kichang Yang, Minho Ryu, Taekyoon Choi, Seungmu Yang, Jiwung Hyun, Sungho Park, Kyubyong Park. A Technical Report for Polyglot-Ko: Open-Source Large-Scale Korean Language Models. https://arxiv.org/abs/2306.02254. [Accessed: 2023].
[75] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever. Language Models are Unsupervised Multitask Learners https://d4mucfpksywv.cloudfront.net/better-language-models/language-models.pdf. [Accessed: 2023].
[76] Oleh Shliazhko, Alena Fenogenova, Maria Tikhonova, Vladislav Mikhailov, Anastasia Kozlova, Tatiana Shavrina. mGPT: Few-Shot Learners Go Multilingual. https://arxiv.org/abs/2204.07580. [Accessed: 2022].

IX. Calculate the Average Characters in a Book

Assuming the following for an average book:

- The book has 250 words per page. https://hotghostwriter.com/blogs/blog/novel-length-how-long-is-long-enough
- Each word is followed by a space. This excludes the first and last word from having a leading or trailing space, respectively.
- The total spaces per page are $250 - 2 = 248$.
- On average, there are 21 new lines per page (assuming 250 words divided by an average of 12 words per line).https://www.myfonts.com/pages/fontscom-learning-fontology-level-2-text-typography-length-column-width
- The average word length is 5.1 characters.https://gitnux.org/average-word-length

The calculation for characters per page is as follows:

$$\begin{aligned}\text{Characters from words} &= \text{Words per page} \times \text{Average word length} \\ &= 250 \times 5.1 \\ &= 1275 \text{ characters} \\ \text{Total characters per page} &= \text{Characters from words} + \text{New lines} + \text{Spaces} \\ &= 1275 + 21 + 248 \\ &= 1544 \text{ characters}\end{aligned}$$

For a book with 355.5 pages, the total character count would be:

$$\begin{aligned}\text{Total character count} &= \text{Characters per page} \times \text{Total pages} \\ &= 1544 \times 355.5 \\ &\approx 548892\end{aligned}$$

This is then rounded to approximately 549,000 characters for the entire book.

X. Calculate the dollar amount per token

To calculate the dollar amount per token and estimate costs for the generation of tokens using OpenAI's GPT-4-32k, you would use the price per token provided by OpenAI at the given time. According to the information provided, OpenAI charges $120.00 per 1 million tokens. For a single token, the cost can be calculated by dividing the total price by the number of tokens:

$$\text{Cost Per Token} = \frac{\text{Total Price}}{\text{Number of Tokens}} \tag{3}$$

Substituting the values:

$$\text{Cost Per Token} = \frac{120}{1,000,000} = 0.00012 \text{ USD} \tag{4}$$

For further details on OpenAI's pricing, refer to their pricing page.

XI. Findings

A. Overall Performance

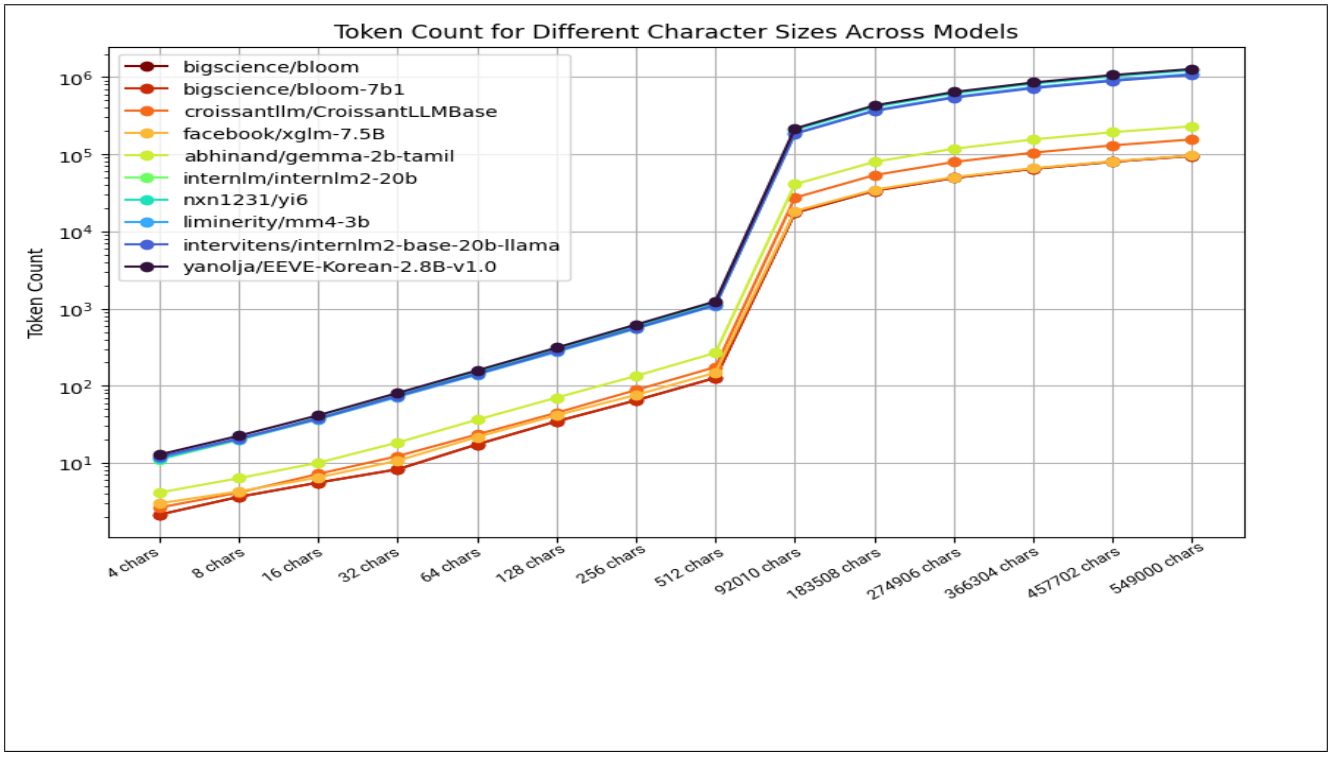


Fig. 1: Token count for different character sizes across models

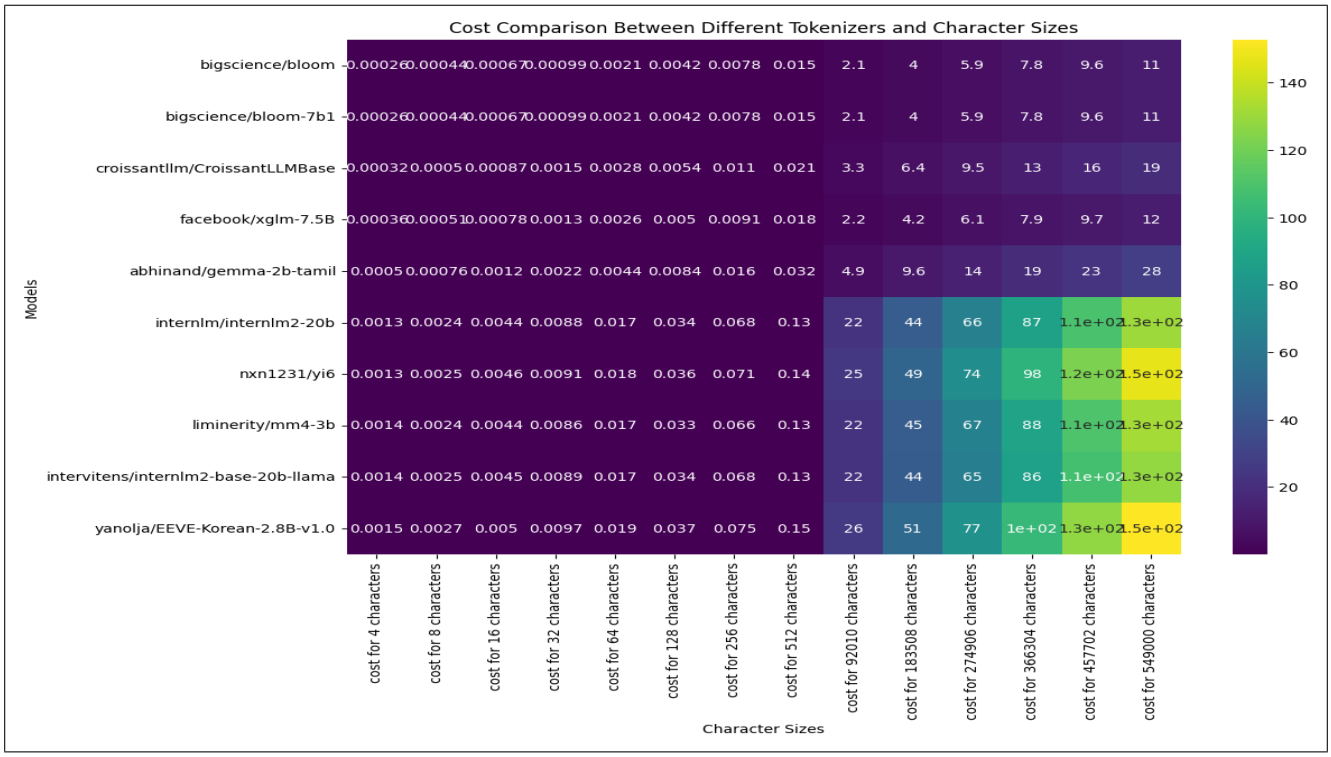


Fig. 2: Cost comparison between different tokenizers and character sizes

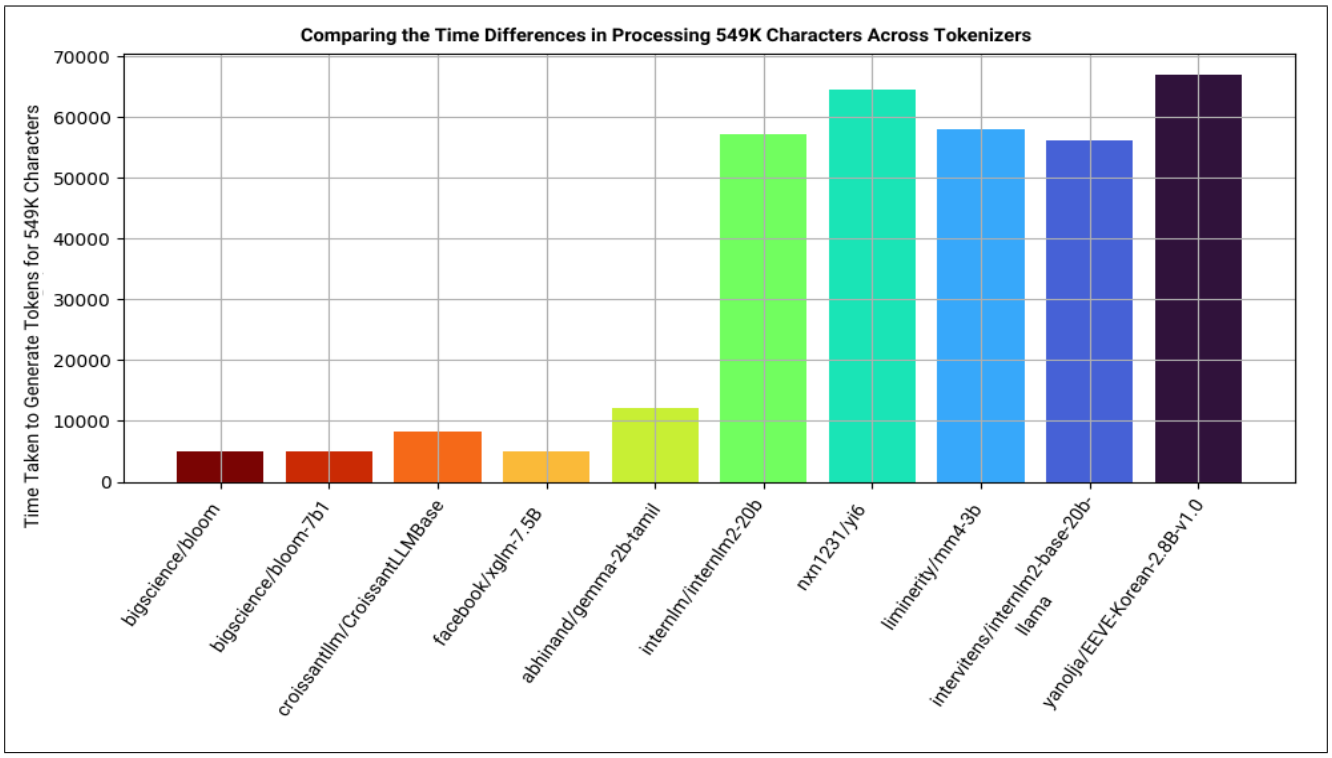


Fig. 3: Comparing the time differences in processing 549K characters across tokenizers

TABLE I: Tokenizer performance metrics for token sizes across various tokenizers for 549,000 characters.

Model	Tokens	Cost ($)	Throughput in sec
bigscience/bloom	95211.50	11.43	5011.13
bigscience/bloom-7b1	95211.50	11.43	5011.13
croissantllm/CroissantLLMBase	155767.28	18.69	8198.28
facebook/xglm-7.5B	96127.89	11.54	5059.36
abhinand/gemma-2b-tamil	230490.61	27.66	12131.08
google/gemma-2b	230490.61	27.66	12131.08
google/gemma-7b	230490.61	27.66	12131.08
instructkr/ko-wand-136M	304604.47	36.55	16031.81
Qwen/Qwen1.5-0.5B	604955.71	72.59	31839.77
Qwen/Qwen1.5-14B	604955.71	72.59	31839.77
Qwen/Qwen1.5-4B	604955.71	72.59	31839.77
Qwen/Qwen1.5-72B	604955.71	72.59	31839.77
Qwen/Qwen1.5-7B	604955.71	72.59	31839.77
Qwen/Qwen1.5-7B-Chat	604955.71	72.59	31839.77
sail/Sailor-4B	604955.71	72.59	31839.77
sail/Sailor-7B	604955.71	72.59	31839.77
ai-forever/mGPT	600206.67	72.02	31589.82
rinna/bilingual-gpt-neox-4b-8k	476506.50	57.18	25079.29
augmxnt/shisa-base-7b-v1	477412.69	57.29	25126.98
NYTK/PULI-GPTrio	648593.14	77.83	34136.48
Dampish/StellarX-4B-V0	706876.62	84.83	37204.03
EleutherAI/gpt-neox-20b	706876.62	84.83	37204.03
EleutherAI/pythia-12b-deduped	706876.62	84.83	37204.03
EleutherAI/pythia-6.9b-deduped	706876.62	84.83	37204.03
OEvortex/vortex-3b-v2	706876.62	84.83	37204.03
matsuo-lab/weblab-10b	706876.62	84.83	37204.03
stabilityai/stablelm-base-alpha-7b-v2	706876.62	84.83	37204.03
togethercomputer/RedPajama-INCITE-7B-Base	706876.62	84.83	37204.03
togethercomputer/RedPajama-INCITE-Base-3B-v1	706876.62	84.83	37204.03
wtang06/mpt-125m-c4	706876.62	84.83	37204.03
stabilityai/stablelm-2-1-6b	704285.80	84.51	37067.67
bigcode/starcoder	763541.39	91.62	40186.39
BAAI/Aquila2-34B	721492.58	86.58	37973.29
KnutJaegersberg/Qwen-14B-Llamafied	735242.53	88.23	38696.98
Weyaxi/Qwen-72B-Llama	735242.53	88.23	38696.98
qnguyen3/quan-1.8b-base	735242.53	88.23	38696.98
andysalerno/mistral-sft-v3	718430.70	86.21	37812.14
internlm/internlm-20b	784610.32	94.15	41295.28
team-lucid/mptk-1b	815419.86	97.85	42916.83
h2oai/h2o-danube-1.8b-base	807739.33	96.93	42512.60
KnutJaegersberg/internlm-20b-llama	767410.62	92.09	40390.03
Deci/DeciLM-7B	698595.70	83.83	36768.19
HIT-SCIR/Chinese-Mixtral-8x7B	698595.70	83.83	36768.19
Replete-AI/Mistral-Evolved-11b-v0.1	698595.70	83.83	36768.19
TencentARC/Mistral-Pro-8B-v0.1	698595.70	83.83	36768.19
ZySec-AI/ZySec-7B	698595.70	83.83	36768.19
awnr/Mistral-7B-v0.1-half-naive-A	698595.70	83.83	36768.19
gordicaleksa/YugoGPT	698595.70	83.83	36768.19
hfl/chinese-mixtral	698595.70	83.83	36768.19
mistralai/Mistral-7B-v0.1	698595.70	83.83	36768.19
mistralai/Mixtral-8x7B-v0.1	698595.70	83.83	36768.19
scb10x/typhoon-7b	698595.70	83.83	36768.19
unsloth/mistral-7b-v0.2	698595.70	83.83	36768.19
upstage/SOLAR-10.7B-v1.0	698595.70	83.83	36768.19

Continued on next page

Table I continued from previous page

Model	Tokens	Cost ($)	Throughput in sec
yanolja/KoSOLAR-10.7B-v0.2	698595.70	83.83	36768.19
itsliupeng/openllama-7b-base	896940.16	107.63	47207.38
itsliupeng/openllama-7b-icl	896940.16	107.63	47207.38
GeneZC/MiniMA-3B	787176.82	94.46	41430.36
IDEA-CCNL/Ziya2-13B-Base	787176.82	94.46	41430.36
LLM360/Amber	787176.82	94.46	41430.36
NucleusAI/nucleus-22B-token-500B	787176.82	94.46	41430.36
TheBloke/Llama-2-7B-GPTQ	787176.82	94.46	41430.36
TinyLlama/TinyLlama-1.1B-intermediate-step-1431k-3T	787176.82	94.46	41430.36
Wanfq/FuseLLM-7B	787176.82	94.46	41430.36
budecosystem/boomer-1b	787176.82	94.46	41430.36
codellama/CodeLlama-34b-hf	787176.82	94.46	41430.36
codellama/CodeLlama-70b-Instruct-hf	787176.82	94.46	41430.36
codellama/CodeLlama-70b-Python-hf	787176.82	94.46	41430.36
codellama/CodeLlama-70b-hf	787176.82	94.46	41430.36
duoqi/Nanbeige-16B-Base-Llama	787176.82	94.46	41430.36
huggyllama/llama-13b	787176.82	94.46	41430.36
huggyllama/llama-65b	787176.82	94.46	41430.36
nnethercott/llava-v1.5-7b-hf-vicuna	787176.82	94.46	41430.36
nnethercott/llava-v1.5-7b-vicuna	787176.82	94.46	41430.36
deepseek-ai/deepseek-llm-67b-base	888692.22	106.64	46773.27
deepseek-ai/deepseek-moe-16b-base	888692.22	106.64	46773.27
tiiuae/falcon-40b	1022268.35	122.67	53803.60
tiiuae/falcon-7b	1022268.35	122.67	53803.60
Writer/palmyra-base	403839.32	48.46	21254.70
Writer/palmyra-large	403839.32	48.46	21254.70
openai-community/gpt2	403839.32	48.46	21254.70
EleutherAI/gpt-j-6b	1049055.85	125.89	55213.47
amu/spin-phi2	1049055.85	125.89	55213.47
cerebras/Cerebras-GPT-13B	1049055.85	125.89	55213.47
cerebras/Cerebras-GPT-6.7B	1049055.85	125.89	55213.47
microsoft/phi-1-5	1049055.85	125.89	55213.47
microsoft/phi-2	1049055.85	125.89	55213.47
beomi/KoRWKV-6B	1143498.76	137.22	60184.15
facebook/opt-30b	1028046.62	123.37	54107.72
facebook/opt-66b	1028046.62	123.37	54107.72
yec019/fbopt-350m-8bit	1028046.62	123.37	54107.72
01-ai/Yi-34B	1247859.42	149.74	65676.81
01-ai/Yi-6B	1247859.42	149.74	65676.81
01-ai/Yi-6B-200K	1247859.42	149.74	65676.81
chargoddard/Yi-34B-Llama	1247859.42	149.74	65676.81
hon9kon9ize/CantoneseLLM-6B-preview202402	1247859.42	149.74	65676.81
AI-Sweden-Models/gpt-sw3-40b	1146138.54	137.54	60323.08
AI-Sweden-Models/gpt-sw3-6.7b	1146138.54	137.54	60323.08
AI-Sweden-Models/gpt-sw3-6.7b-v2	1146138.54	137.54	60323.08
TurkuNLP/gpt3-finnish-13B	1316390.97	157.97	69283.74
EleutherAI/polyglot-ko-12.8b	1311993.48	157.44	69052.29
internlm/internlm2-20b	1086734.21	130.41	57196.54
nxn1231/yi6	1227704.38	147.32	64616.02
liminerity/mm4-3b	1100313.23	132.04	57911.22
intervitens/internlm2-base-20b-llama	1068363.15	128.20	56229.64
yanolja/EEVE-Korean-2.8B-v1.0	1273168.90	152.78	67008.89

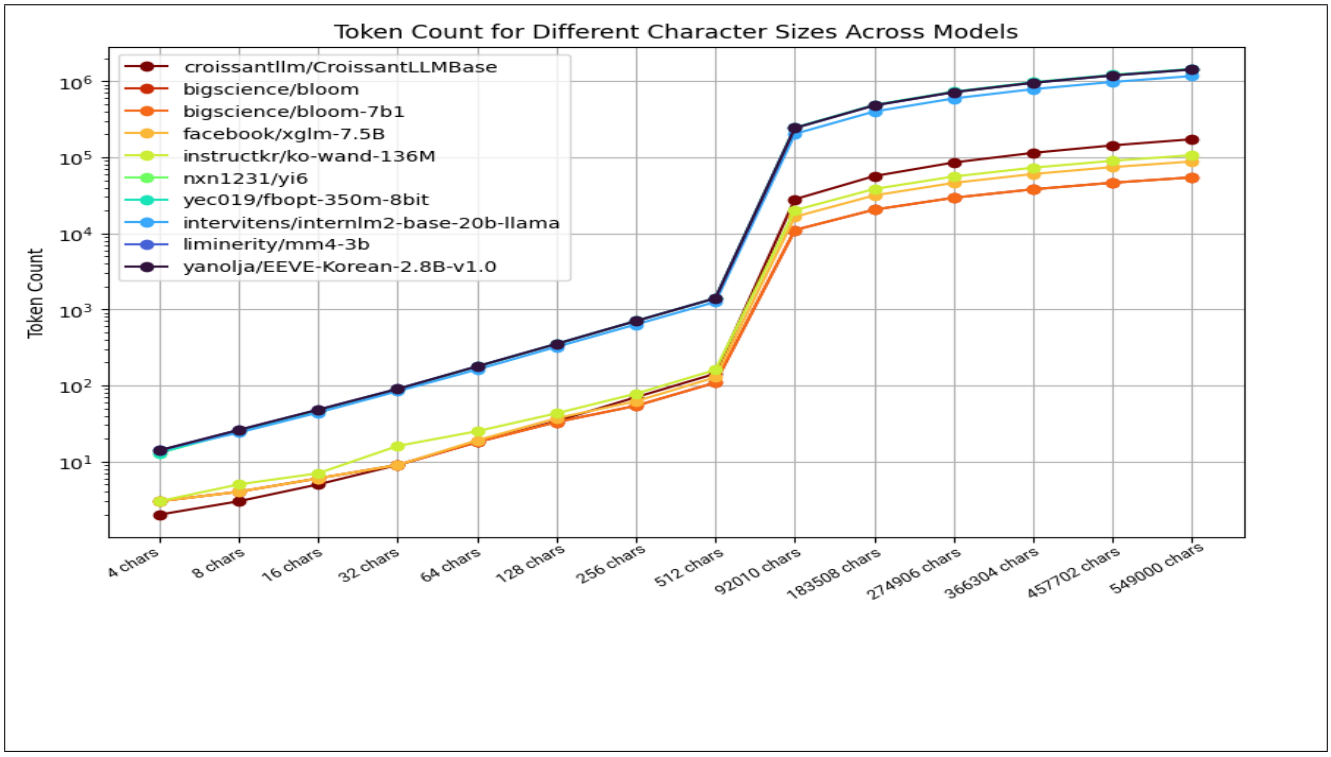


Fig. 4: Token count for different character sizes across models (Tamil)

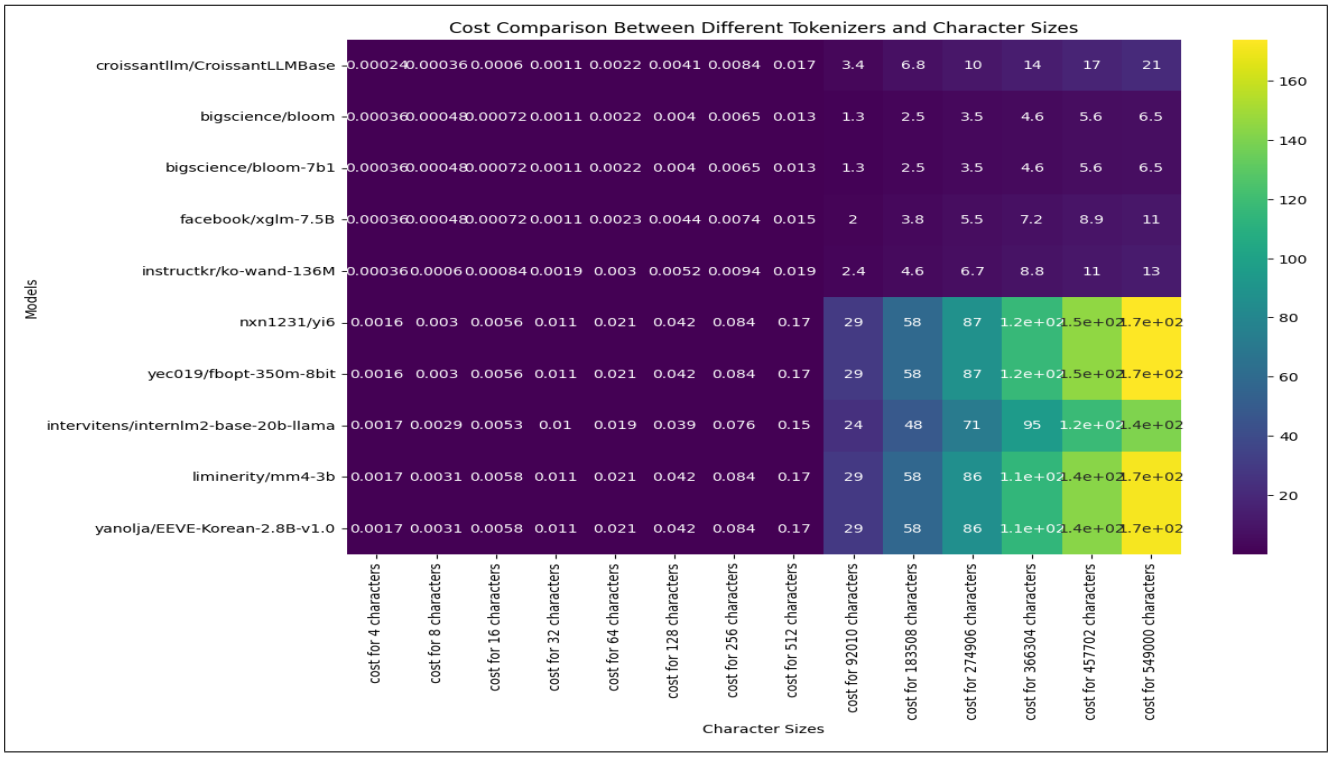


Fig. 5: Cost comparison between different tokenizers and character sizes (Tamil)

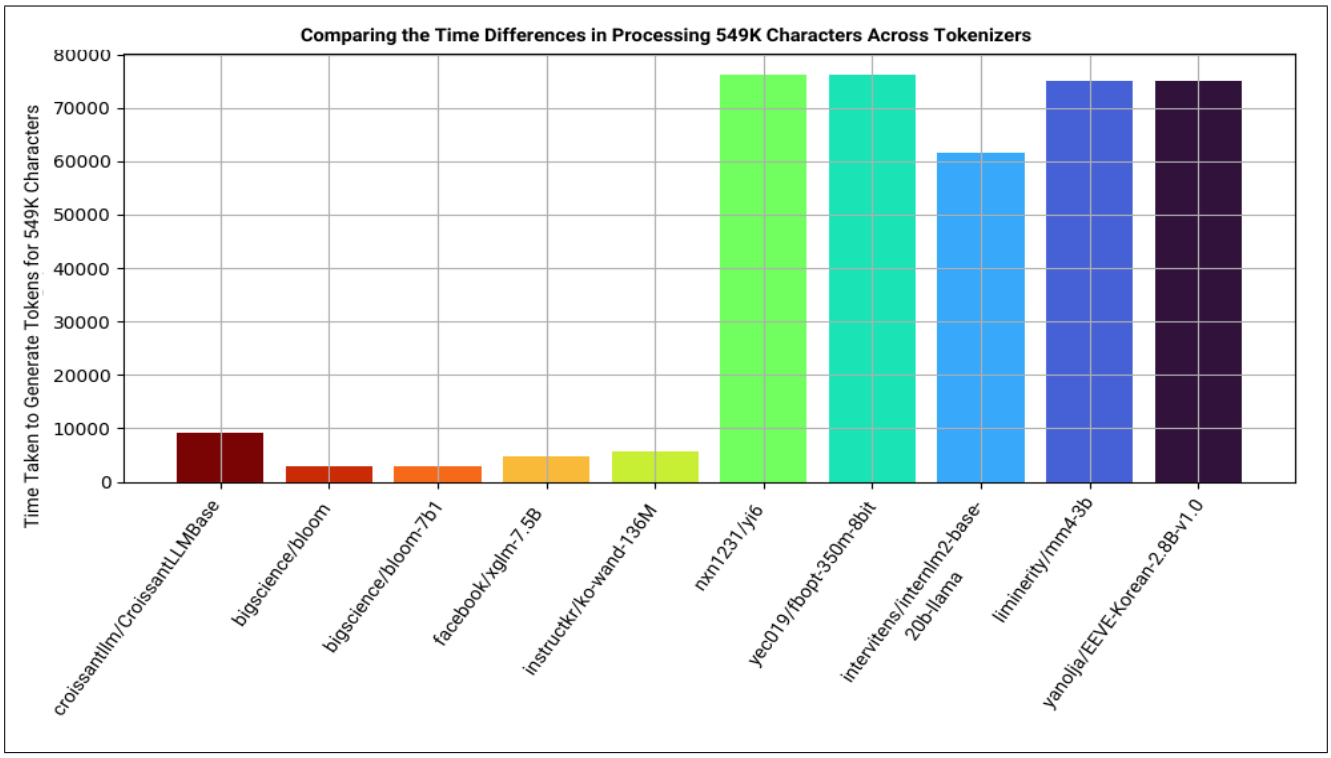


Fig. 6: Comparing the time differences in processing 549K characters across tokenizers (Tamil)

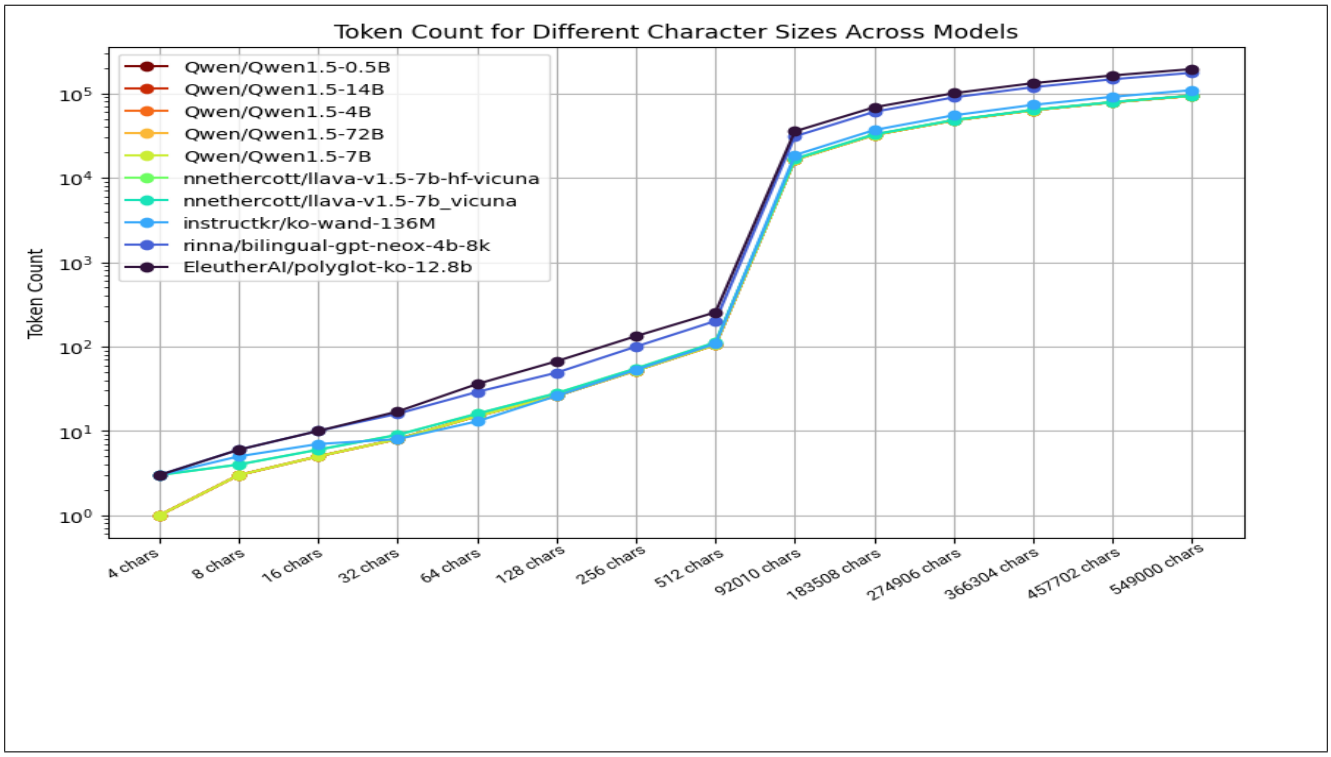


Fig. 7: Token count for different character sizes across models (English)

TABLE II: Tokenizer performance metrics for token sizes across various tokenizers for 549,000 characters in Tamil language.

Model	**Tokens**	**Cost ($)**	**Throughput in sec**
bigscience/bloom	54488.43	6.54	2867.81
bigscience/bloom-7b1	54488.43	6.54	2867.81
facebook/xglm-7.5B	88185.35	10.58	4641.33
instructkr/ko-wand-136M	106619.59	12.79	5611.56
croissantllm/CroissantLLMBase	172777.25	20.73	9093.54
intervitens/internlm2-base-20b-llama	1171442.40	140.57	61654.86
liminerity/mm4-3b	1427856.50	171.34	75150.34
yanolja/EEVE-Korean-2.8B-v1.0	1427856.50	171.34	75150.34
nxn1231/yi6	1449115.05	173.89	76269.21
yec019/fbopt-350m-8bit	1449115.05	173.89	76269.21

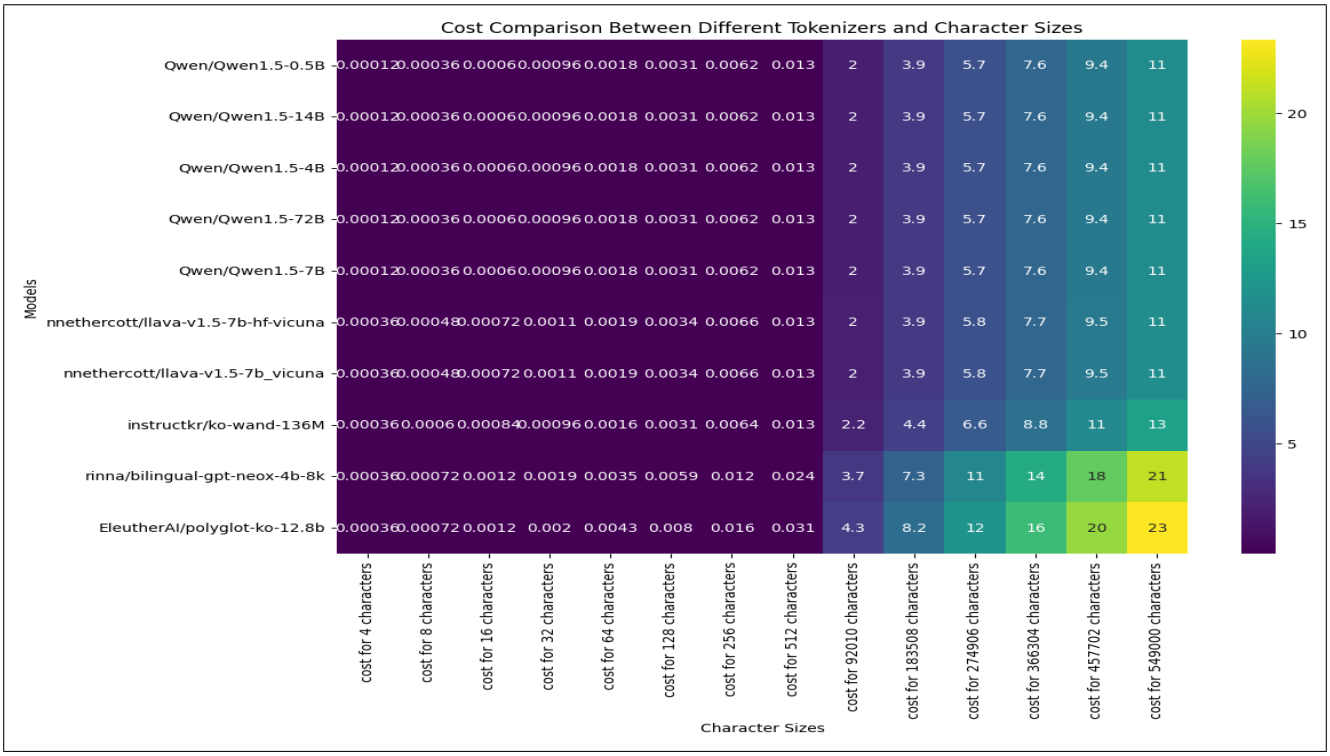


Fig. 8: Cost comparison between different tokenizers and character sizes (English)

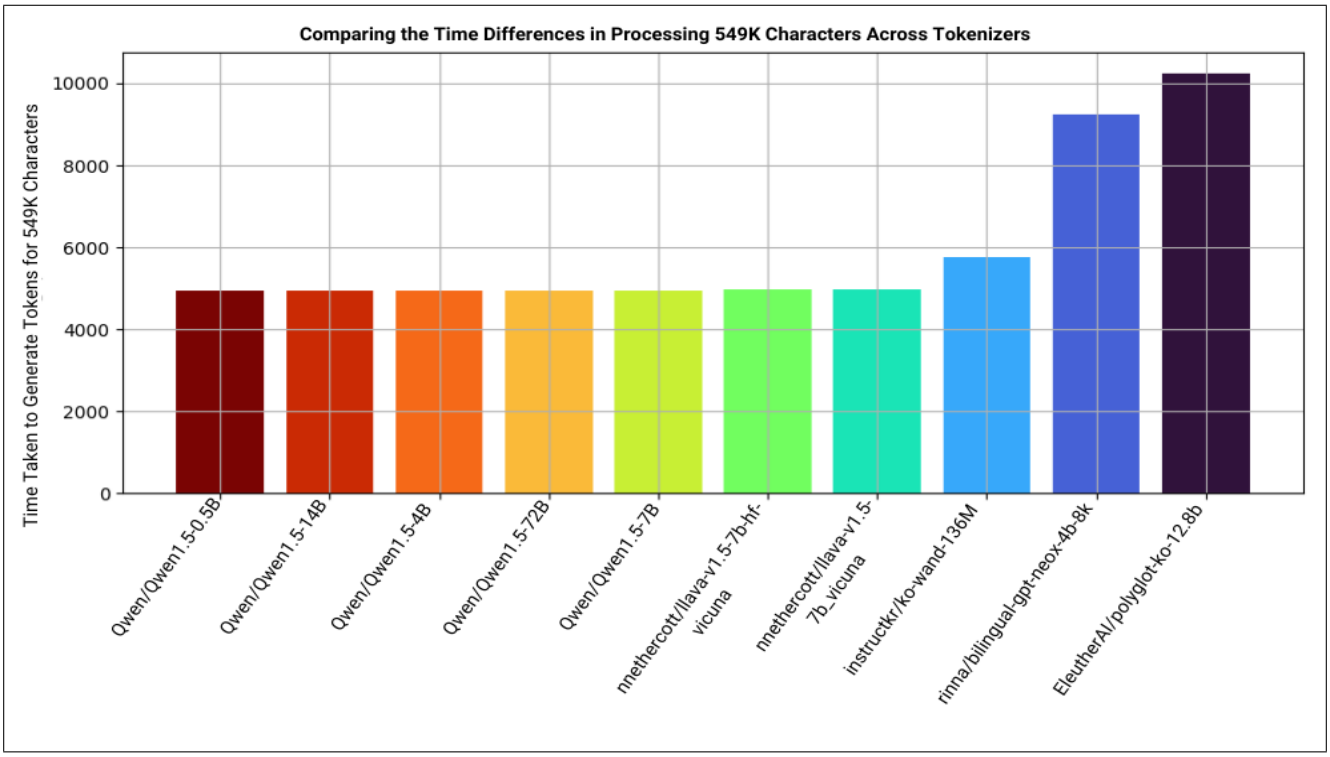


Fig. 9: Comparing the time differences in processing 549K characters across tokenizers (English)

TABLE III: Tokenizer performance metrics for token sizes across various tokenizers for 549,000 characters in English language.

Model	Tokens	Cost ($)	Throughput in sec
Qwen/Qwen1.5-0.5B	93769.35	11.25	4935.23
Qwen/Qwen1.5-14B	93769.35	11.25	4935.23
Qwen/Qwen1.5-4B	93769.35	11.25	4935.23
Qwen/Qwen1.5-72B	93769.35	11.25	4935.23
Qwen/Qwen1.5-7B	93769.35	11.25	4935.23
nnethercott/llava-v1.5-7b-hf-vicuna	94295.22	11.32	4962.91
nnethercott/llava-v1.5-7b-vicuna	94295.22	11.32	4962.91
instructkr/ko-wand-136M	109395.09	13.13	5757.64
rinna/bilingual-gpt-neox-4b-8k	175824.41	21.10	9253.92
EleutherAI/polyglot-ko-12.8b	194427.88	23.33	10233.05

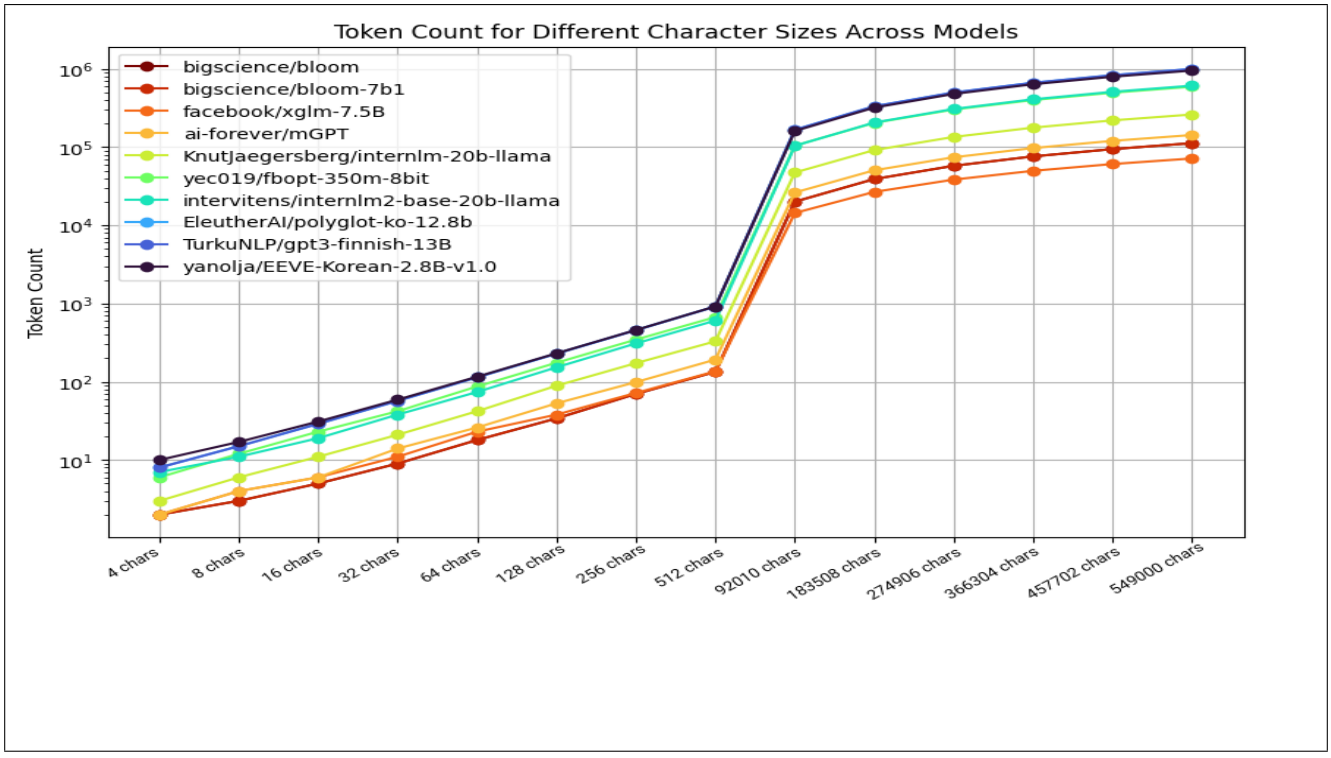


Fig. 10: Token count for different character sizes across models (Urdu)

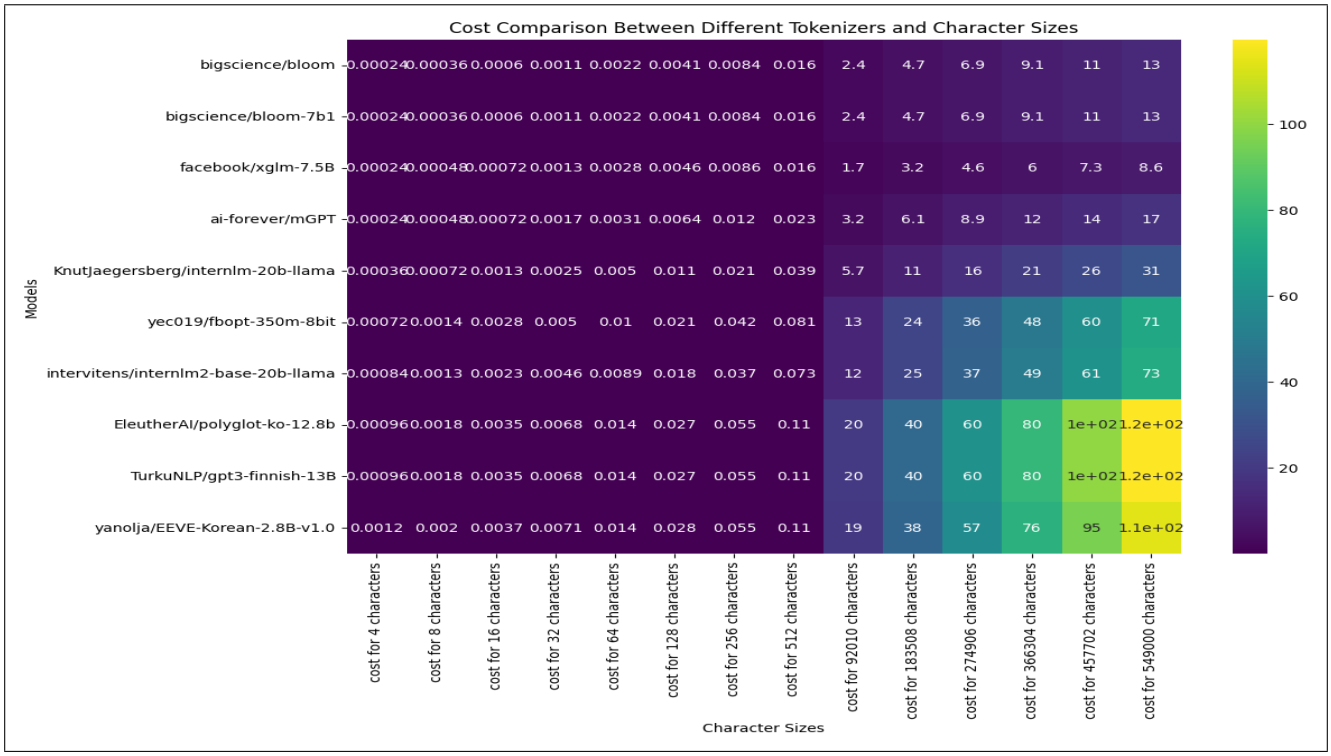


Fig. 11: Cost comparison between different tokenizers and character sizes (Urdu)

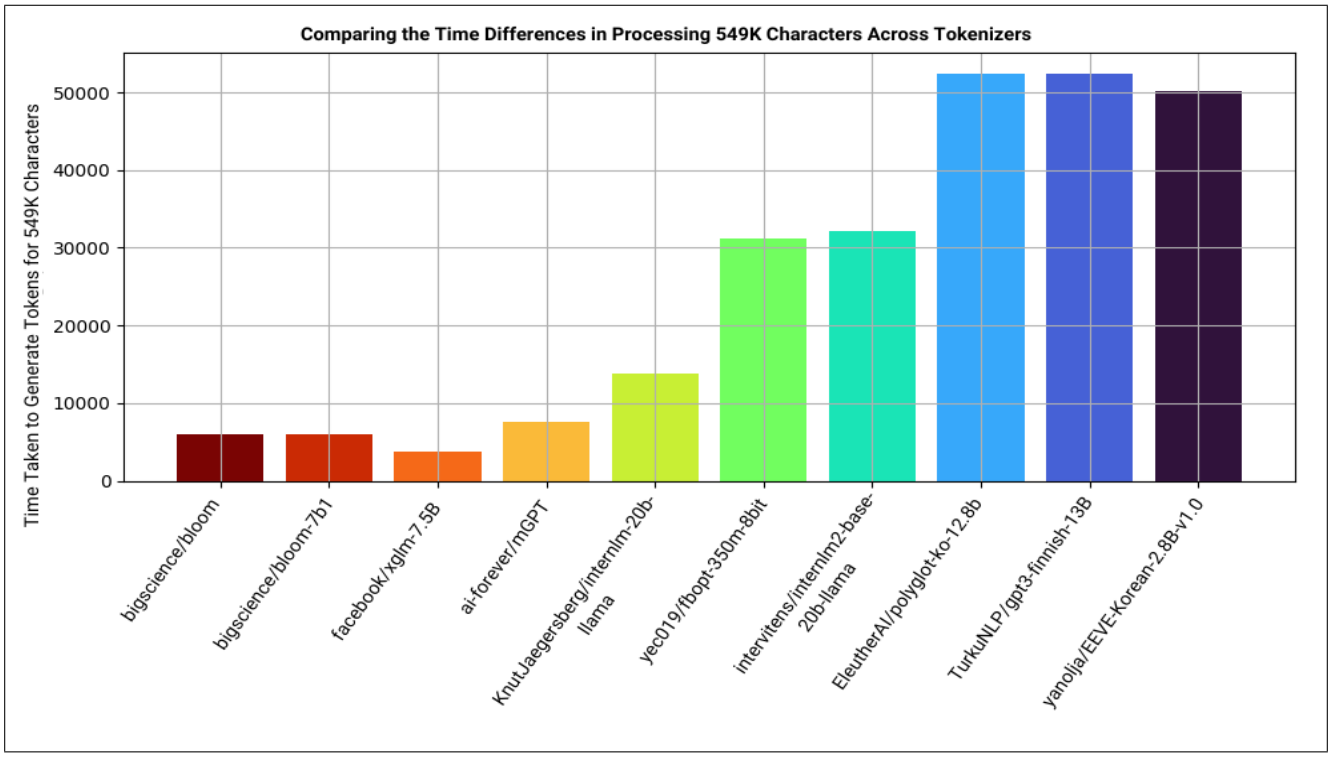


Fig. 12: Comparing the time differences in processing 549K characters across tokenizers (Urdu)

TABLE IV: Tokenizer performance metrics for token sizes across various tokenizers for 549,000 characters in Urdu language.

Model	Tokens	Cost ($)	Throughput in sec
facebook/xglm-7.5B	71538.30	8.58	3765.17
bigscience/bloom	112367.81	13.48	5914.10
bigscience/bloom-7b1	112367.81	13.48	5914.10
ai-forever/mGPT	142740.02	17.13	7512.63
KnutJaegersberg/internlm-20b-llama	261240.90	31.35	13749.52
yec019/fbopt-350m-8bit	591793.90	71.02	31147.05
intervitens/internlm2-base-20b-llama	610339.25	73.24	32123.12
yanolja/EEVE-Korean-2.8B-v1.0	952241.11	114.27	50117.95
EleutherAI/polyglot-ko-12.8b	996487.17	119.58	52446.69
TurkuNLP/gpt3-finnish-13B	996487.17	119.58	52446.69

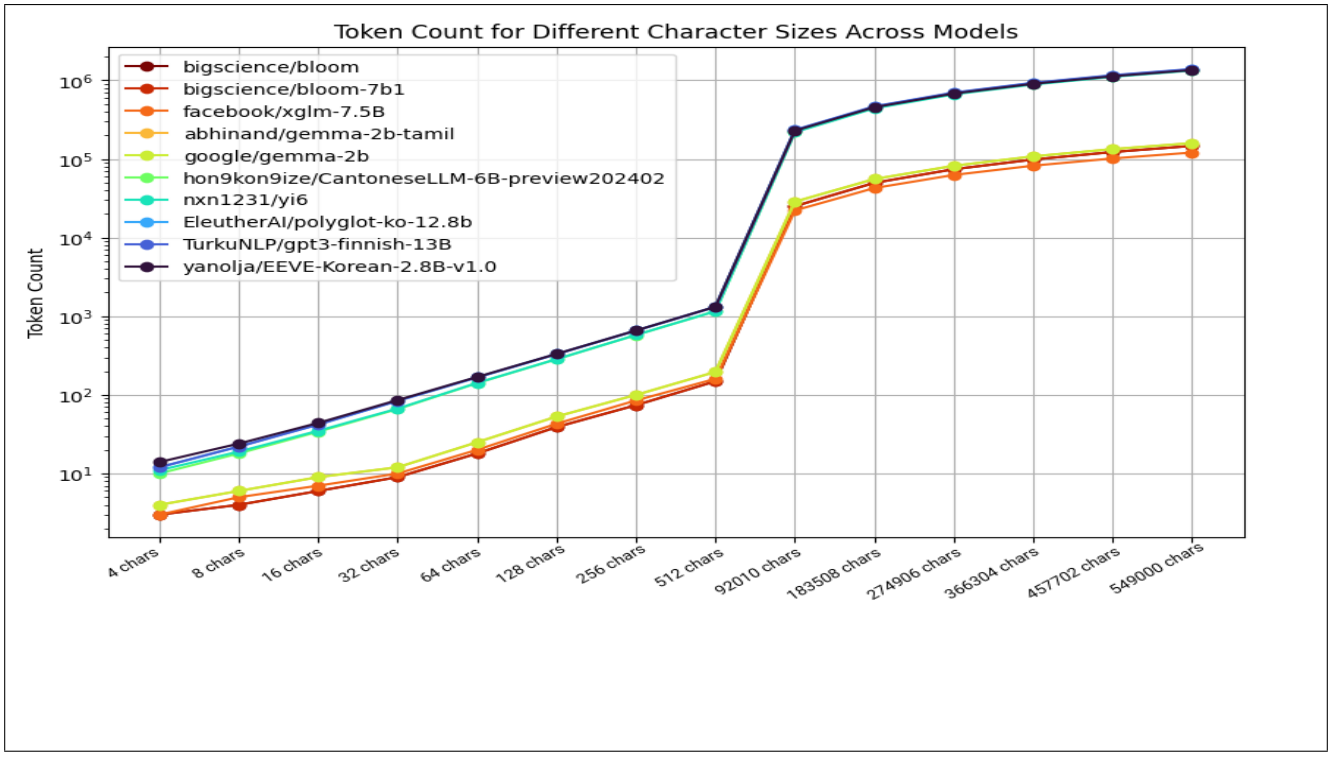


Fig. 13: Token count for different character sizes across models (Hindi)

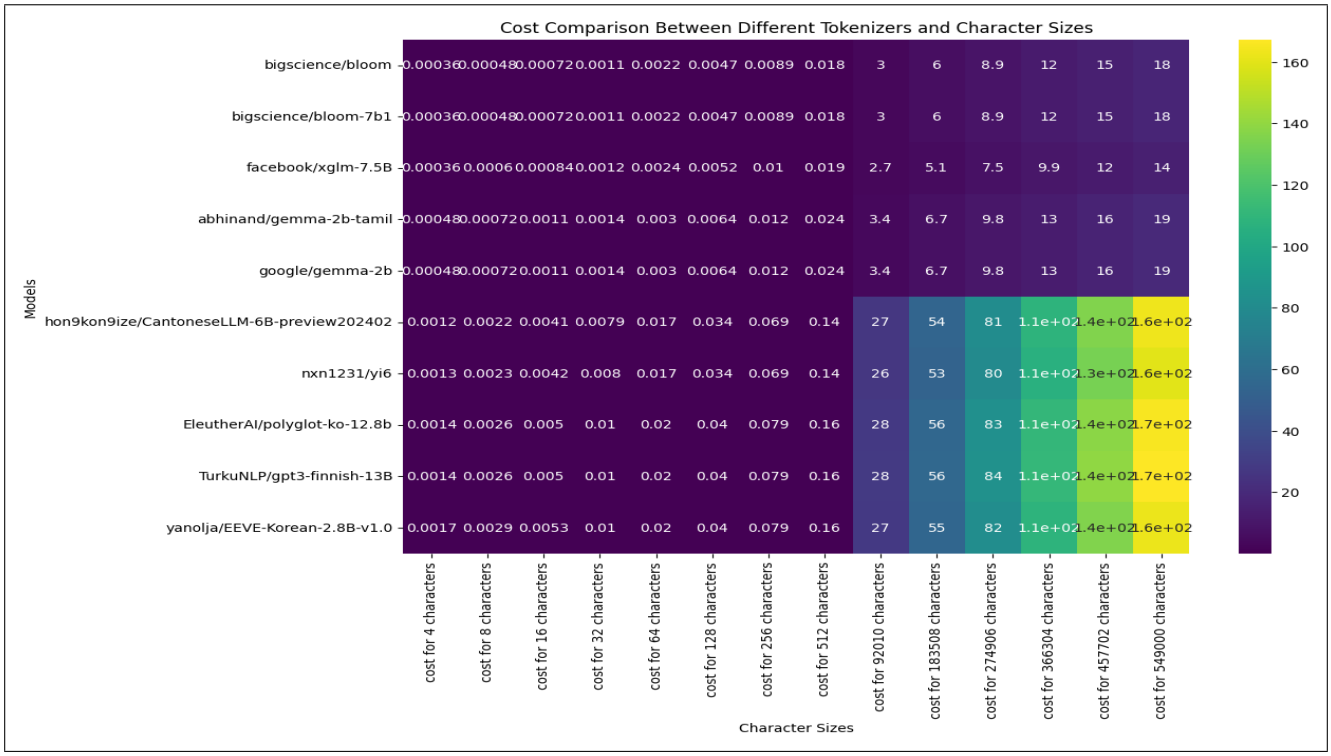


Fig. 14: Cost comparison between different tokenizers and character sizes (Hindi)

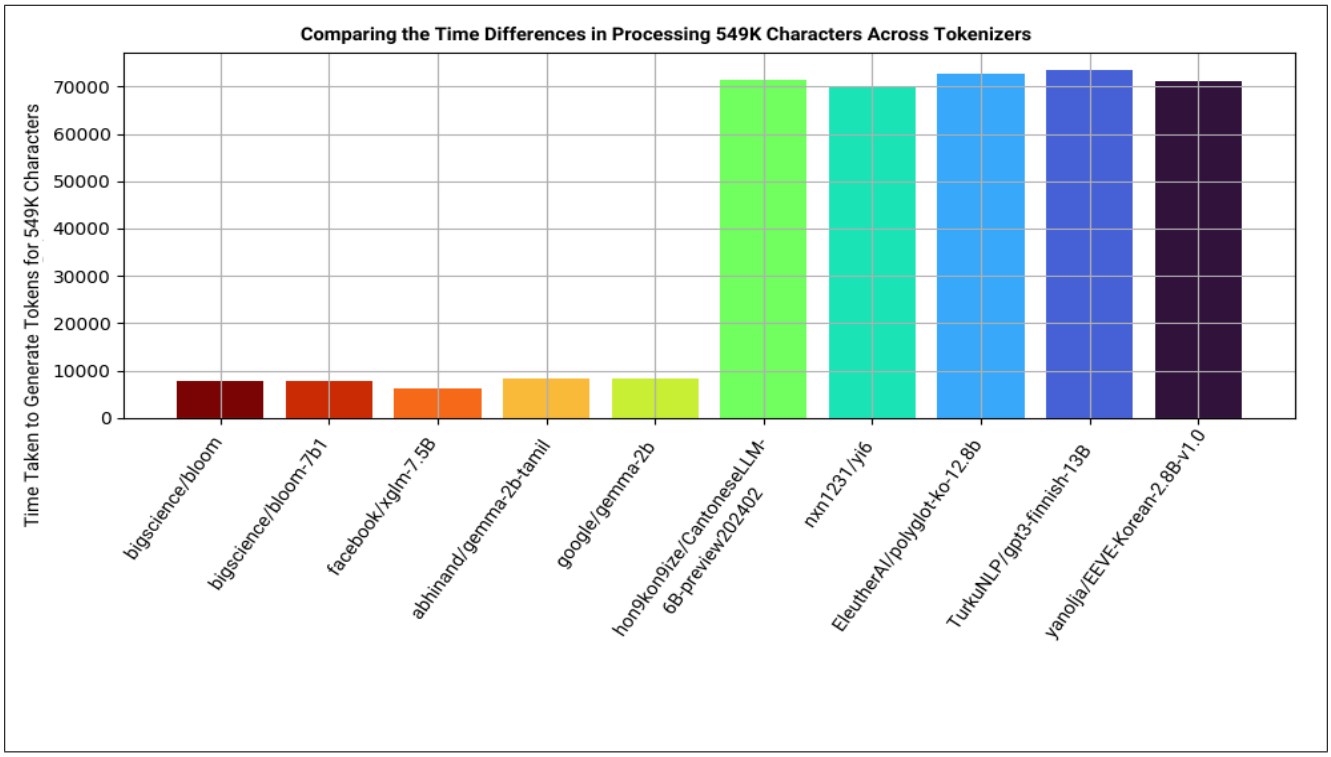


Fig. 15: Comparing the time differences in processing 549K characters across tokenizers (Hindi)

TABLE V: Tokenizer performance metrics for token sizes across various tokenizers for 549,000 characters in Hindi language.

Model	Tokens	Cost ($)	Throughput in sec
facebook/xglm-7.5B	120744.38	14.49	6354.97
bigscience/bloom	146731.17	17.61	7722.69
bigscience/bloom-7b1	146731.17	17.61	7722.69
abhinand/gemma-2b-tamil	159174.74	19.10	8377.62
google/gemma-2b	159174.74	19.10	8377.62
nxn1231/yi6	1332436.93	159.89	70128.26
yanolja/EEVE-Korean-2.8B-v1.0	1350835.75	162.10	71096.62
hon9kon9ize/CantoneseLLM-6B-preview202402	1356758.23	162.81	71408.33
EleutherAI/polyglot-ko-12.8b	1378999.04	165.48	72578.90
TurkuNLP/gpt3-finnish-13B	1394176.46	167.30	73377.71

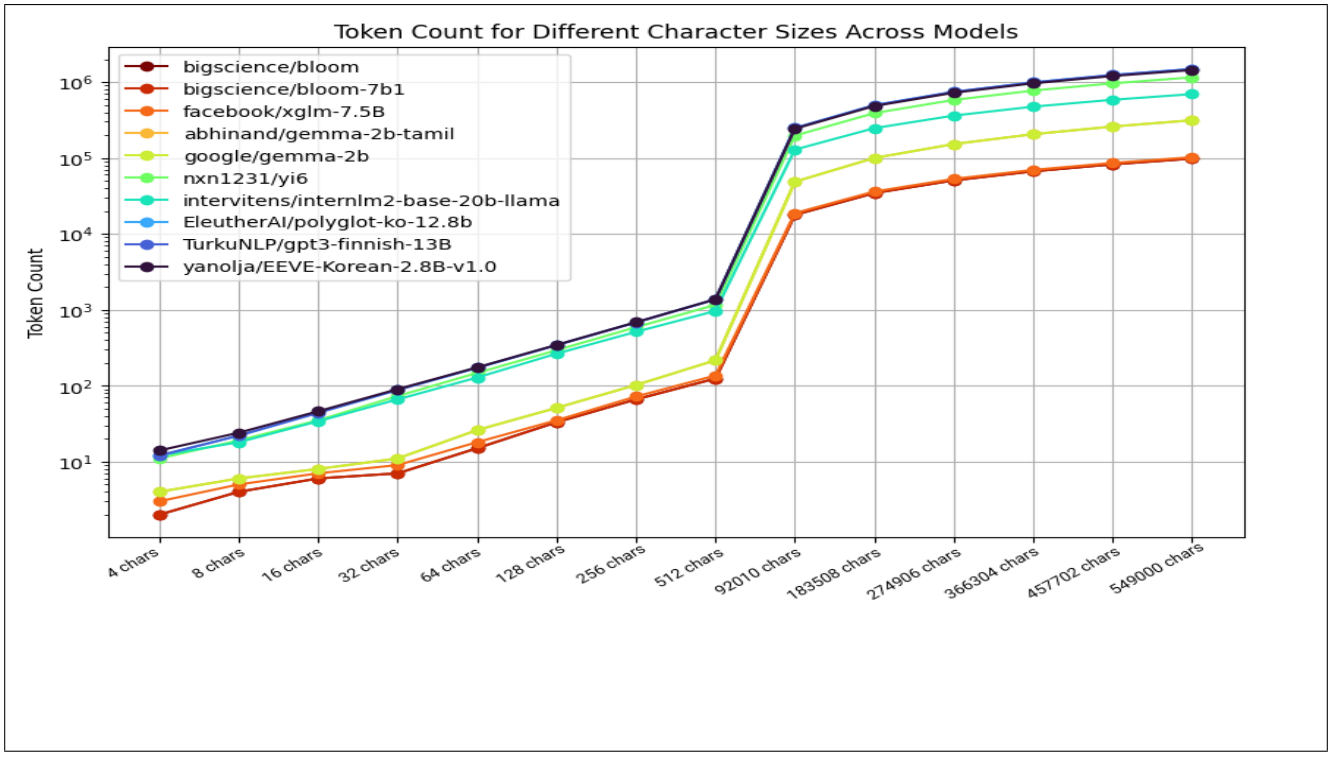


Fig. 16: Token count for different character sizes across models (Nepali)

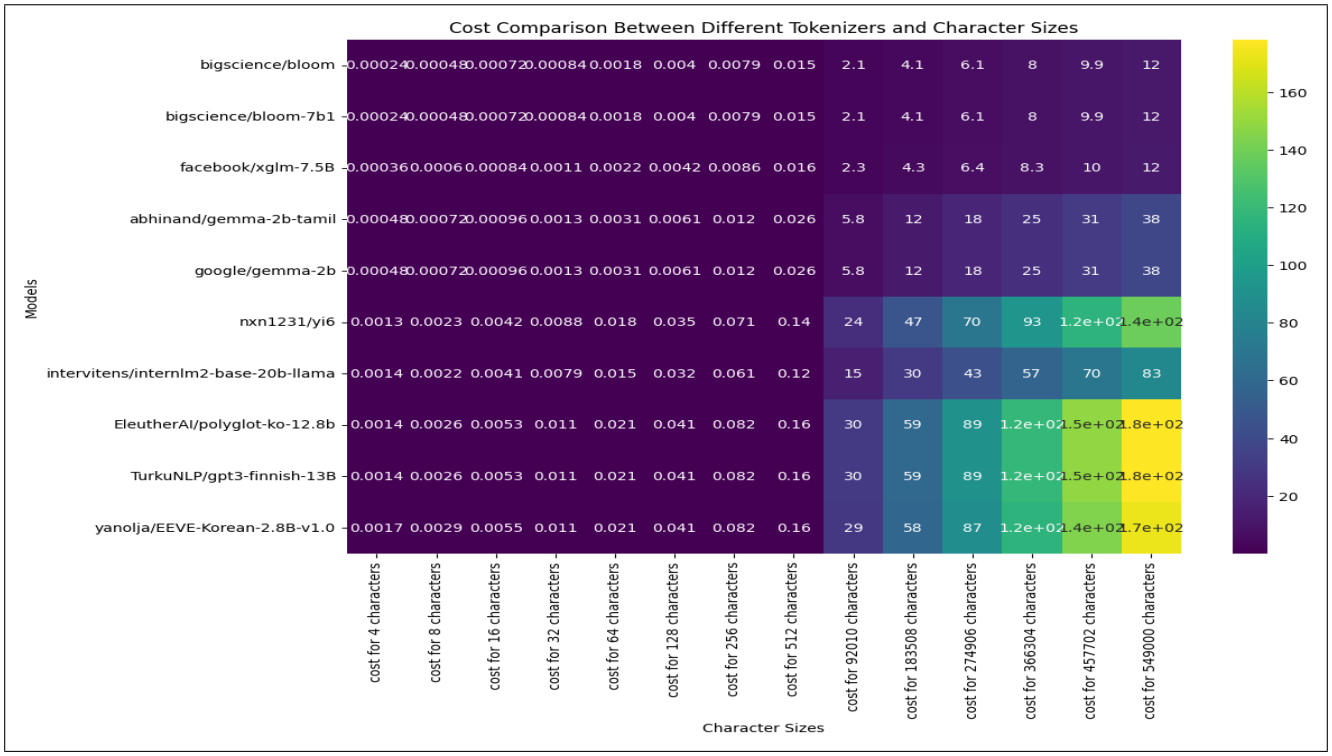


Fig. 17: Cost comparison between different tokenizers and character sizes (Nepali)

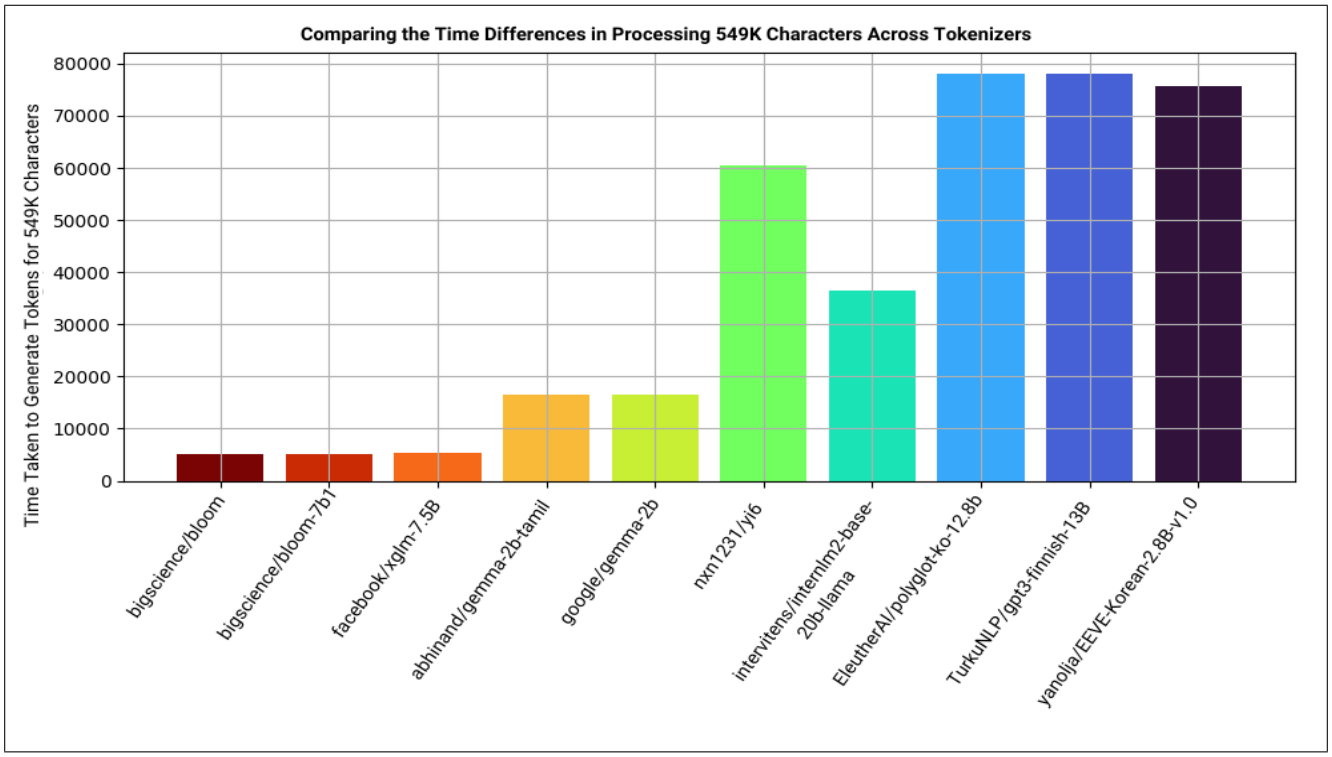


Fig. 18: Comparing the time differences in processing 549K characters across tokenizers (Nepali)

TABLE VI: Tokenizer performance metrics for token sizes across various tokenizers for 549,000 characters in Nepali language.

Model	Tokens	Cost ($)	Throughput in sec
bigscience/bloom	98055.07	11.77	5160.79
bigscience/bloom-7b1	98055.07	11.77	5160.79
facebook/xglm-7.5B	102134.07	12.26	5375.48
abhinand/gemma-2b-tamil	313607.40	37.63	16505.65
google/gemma-2b	313607.40	37.63	16505.65
intervitens/internlm2-base-20b-llama	693205.14	83.18	36484.48
nxn1231/yi6	1151515.87	138.18	60606.10
yanolja/EEVE-Korean-2.8B-v1.0	1439991.66	172.80	75789.03
EleutherAI/polyglot-ko-12.8b	1484564.29	178.15	78134.96
TurkuNLP/gpt3-finnish-13B	1484564.29	178.15	78134.96

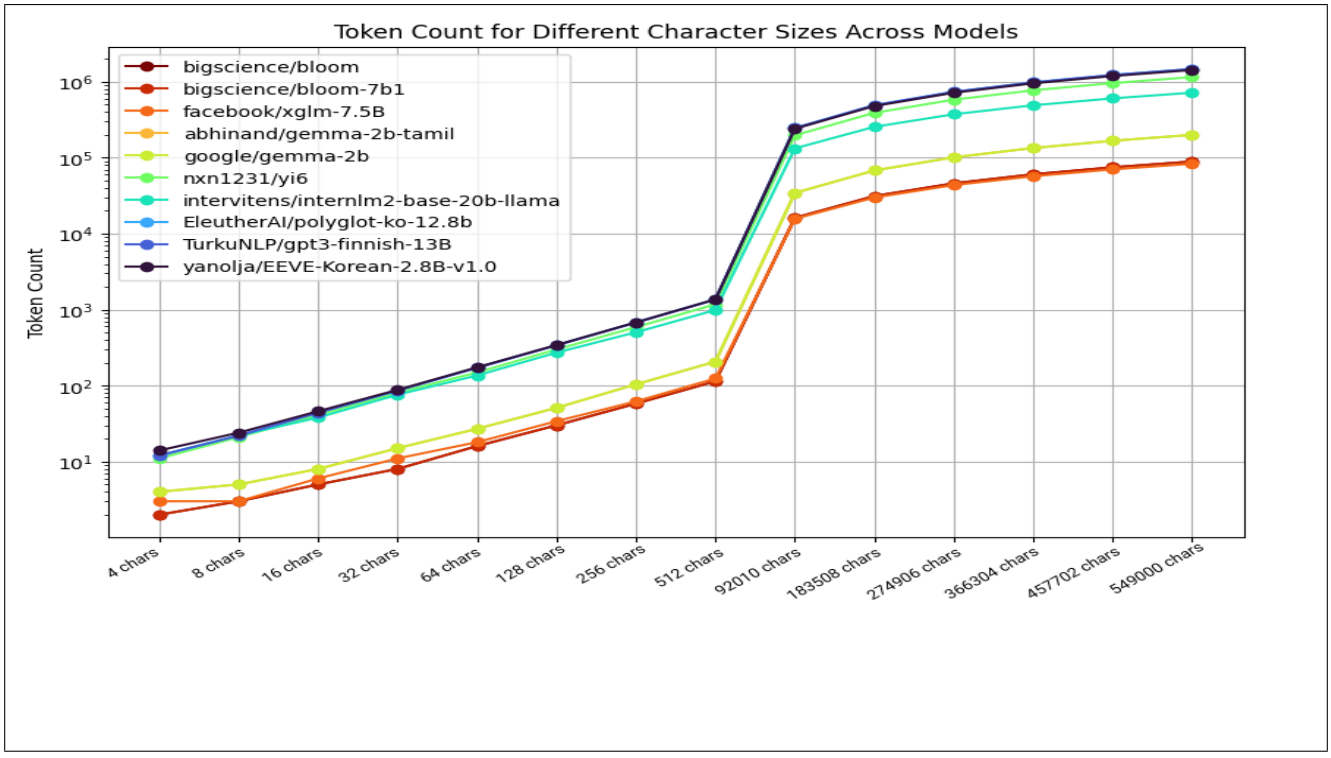


Fig. 19: Token count for different character sizes across models (Marathi)

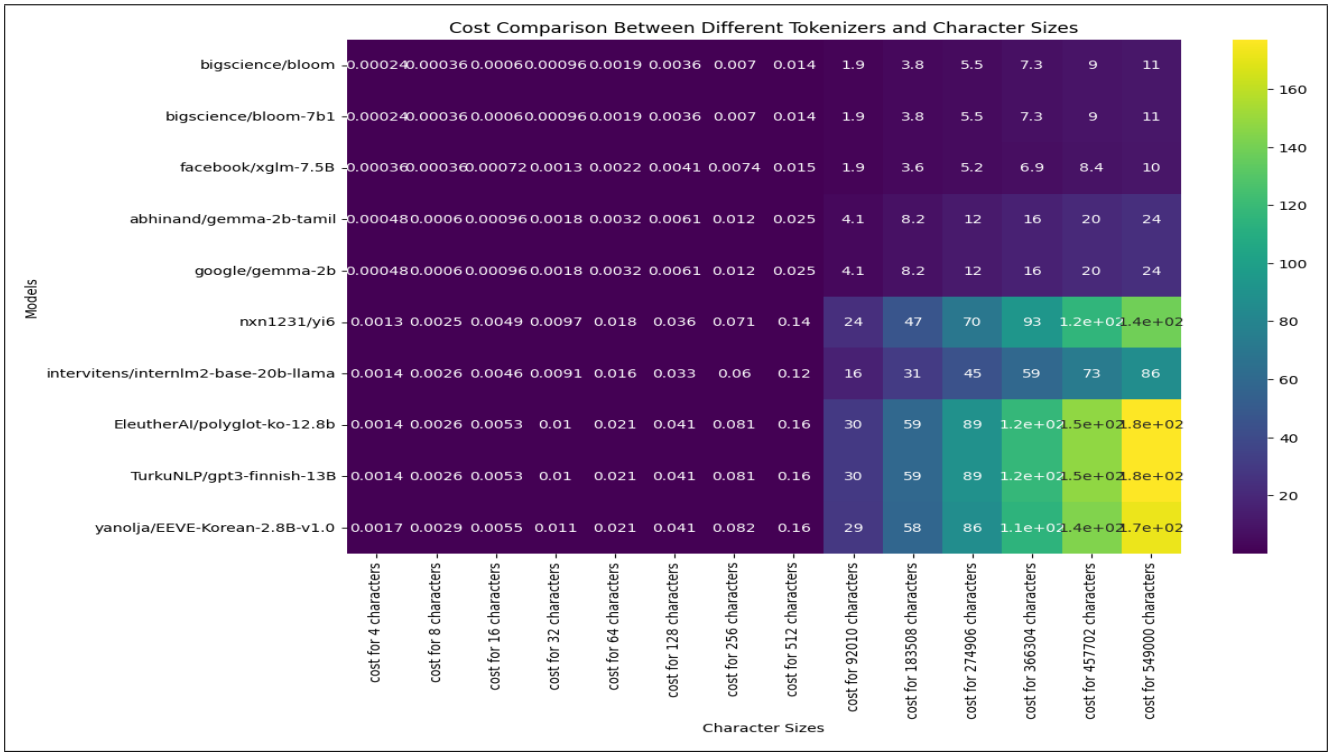


Fig. 20: Cost comparison between different tokenizers and character sizes (Marathi)

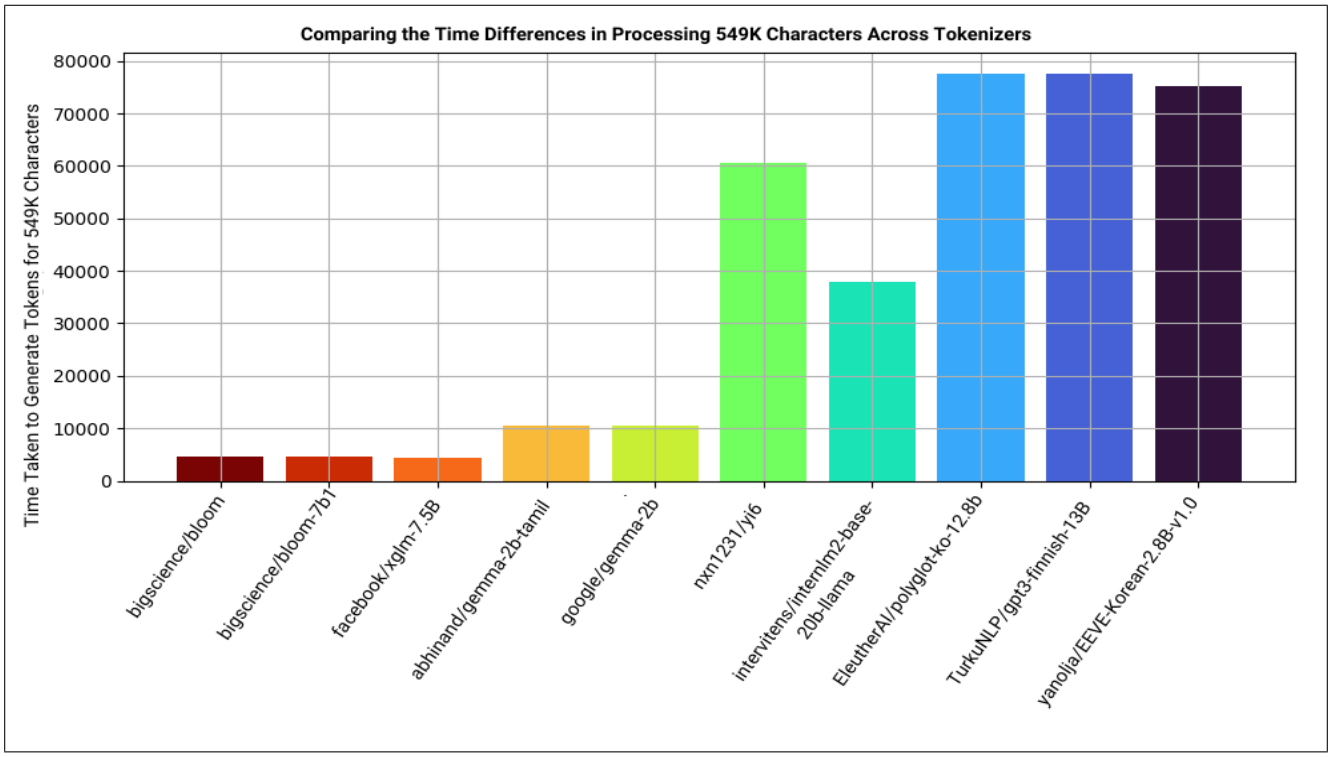


Fig. 21: Comparing the time differences in processing 549K characters across tokenizers (Marathi)

TABLE VII: Tokenizer performance metrics for token sizes across various tokenizers for 549,000 characters in Marathi language.

Model	Tokens	Cost ($)	Throughput in sec
facebook/xglm-7.5B	83408.62	10.01	4389.93
bigscience/bloom	89216.21	10.71	4695.59
bigscience/bloom-7b1	89216.21	10.71	4695.59
abhinand/gemma-2b-tamil	200359.86	24.04	10545.26
google/gemma-2b	200359.86	24.04	10545.26
intervitens/internlm2-base-20b-llama	719180.94	86.30	37851.63
nxn1231/yi6	1152549.48	138.31	60660.50
yanolja/EEVE-Korean-2.8B-v1.0	1430708.07	171.68	75300.42
EleutherAI/polyglot-ko-12.8b	1475242.52	177.03	77644.34
TurkuNLP/gpt3-finnish-13B	1475242.52	177.03	77644.34

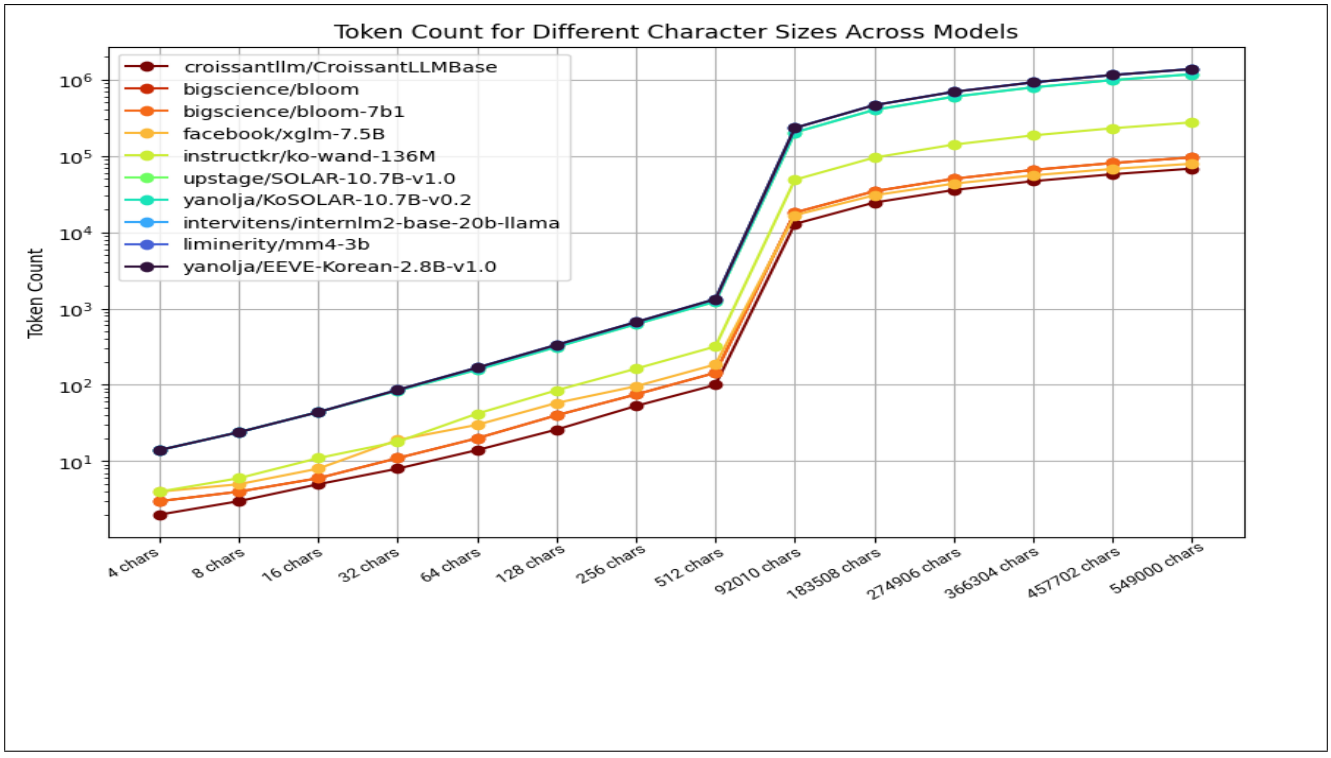


Fig. 22: Token count for different character sizes across models (Punjabi)

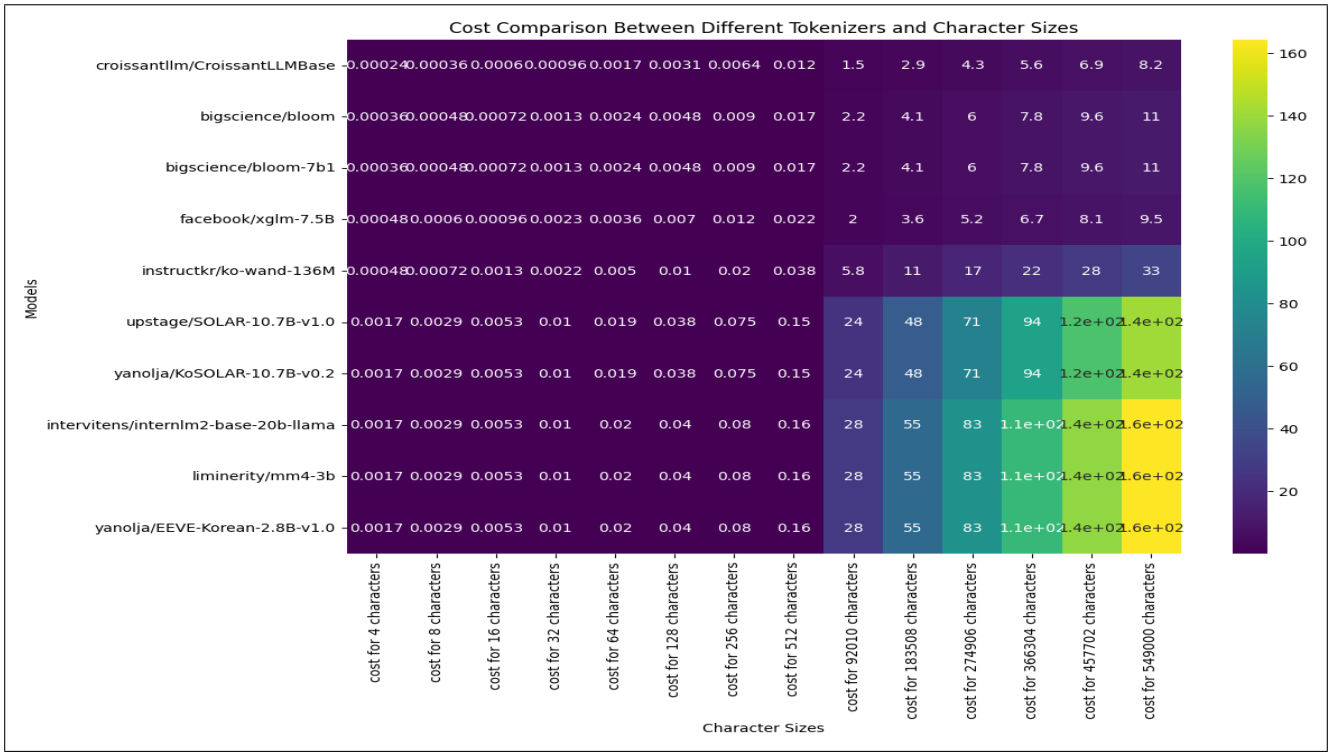


Fig. 23: Cost comparison between different tokenizers and character sizes (Punjabi)

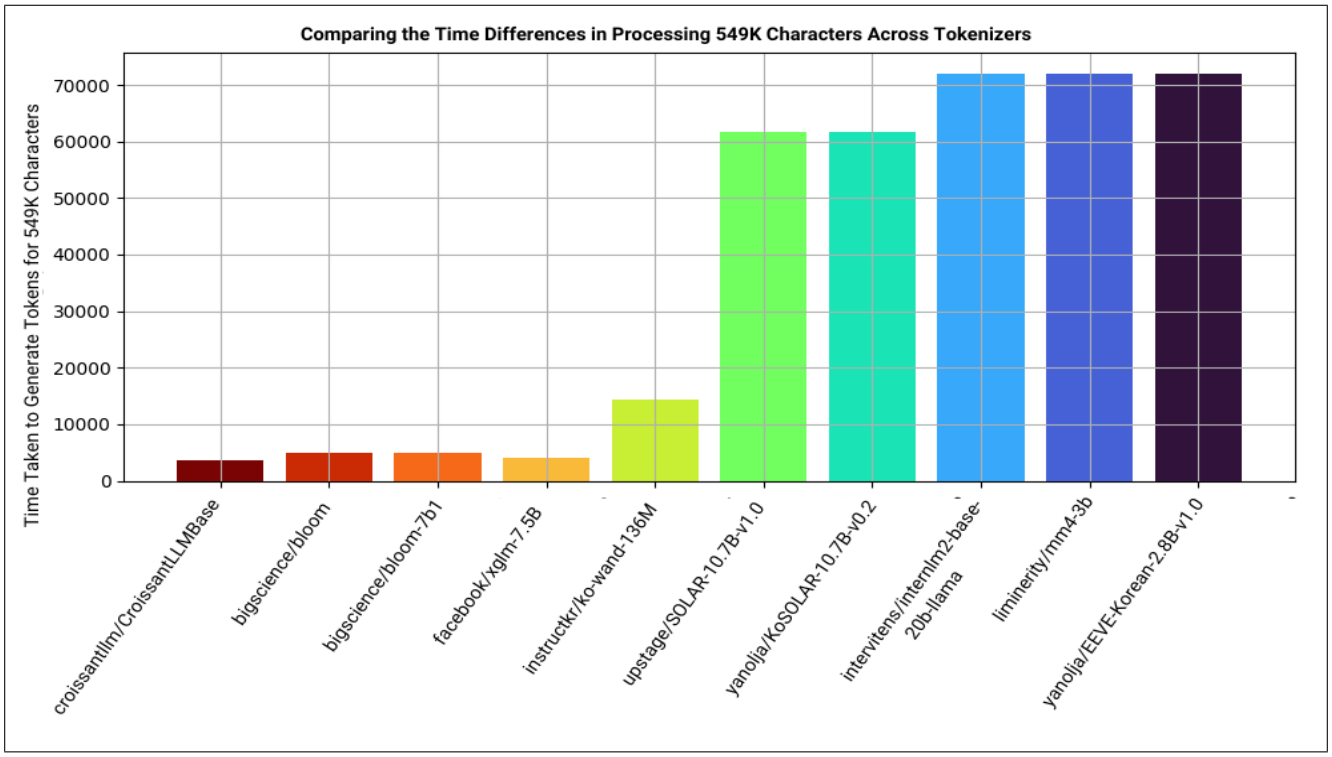


Fig. 24: Comparing the time differences in processing 549K characters across tokenizers (Punjabi)

TABLE VIII: Tokenizer performance metrics for token sizes across various tokenizers for 549,000 characters in Punjabi language.

Model	Tokens	Cost ($)	Throughput in sec
croissantllm/CroissantLLMBase	67955.61	8.15	3576.61
facebook/xglm-7.5B	78791.85	9.46	4146.94
bigscience/bloom	95224.17	11.43	5011.80
bigscience/bloom-7b1	95224.17	11.43	5011.80
instructkr/ko-wand-136M	273787.15	32.85	14409.85
upstage/SOLAR-10.7B-v1.0	1171051.29	140.53	61634.28
yanolja/KoSOLAR-10.7B-v0.2	1171051.29	140.53	61634.28
intervitens/internlm2-base-20b-llama	1368693.74	164.24	72036.51
liminerity/mm4-3b	1368693.74	164.24	72036.51
yanolja/EEVE-Korean-2.8B-v1.0	1368693.74	164.24	72036.51

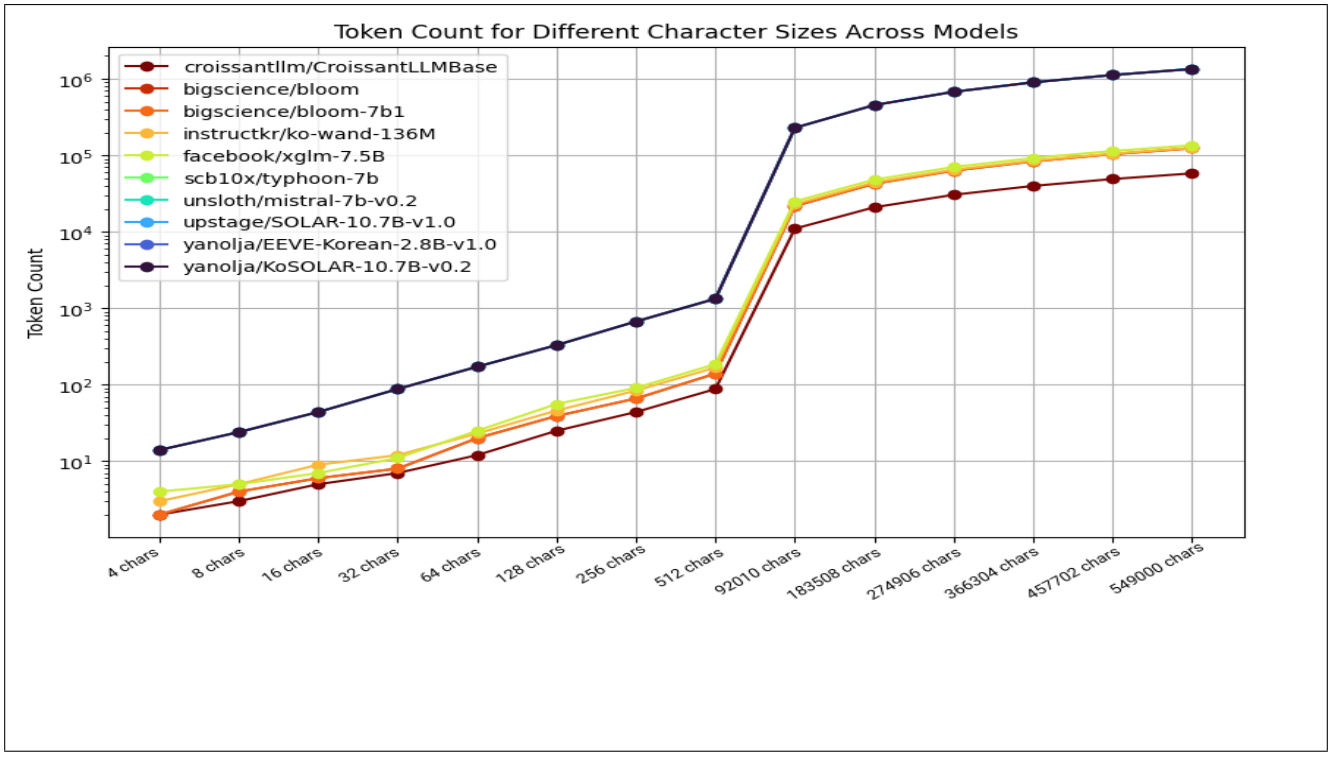


Fig. 25: Token count for different character sizes across models (Oriya)

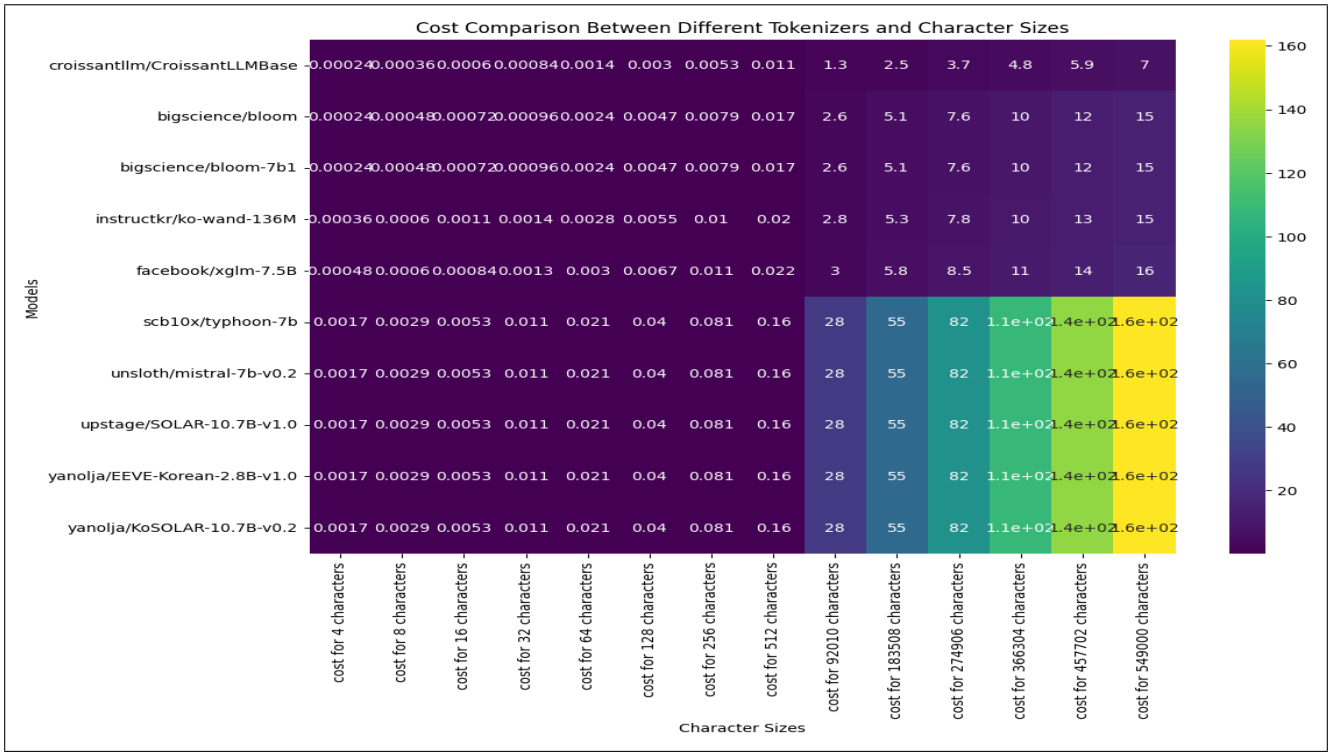


Fig. 26: Cost comparison between different tokenizers and character sizes (Oriya)

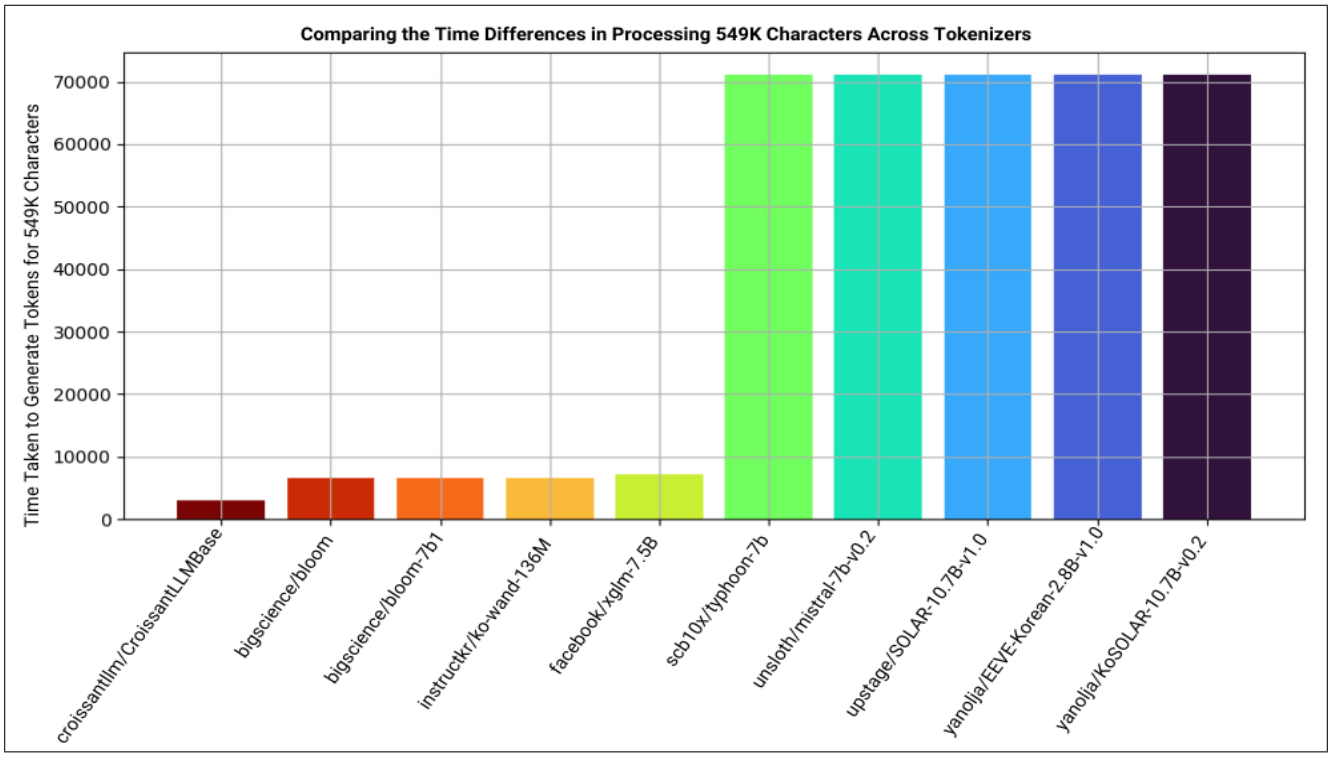


Fig. 27: Comparing the time differences in processing 549K characters across tokenizers (Oriya)

TABLE IX: Tokenizer performance metrics for token sizes across various tokenizers for 549,000 characters in Oriya language.

Model	Tokens	Cost ($)	Throughput in sec
croissantllm/CroissantLLMBase	58188.18	6.98	3062.54
bigscience/bloom	124065.77	14.89	6529.78
bigscience/bloom-7b1	124065.77	14.89	6529.78
instructkr/ko-wand-136M	125397.06	15.05	6599.85
facebook/xglm-7.5B	135678.78	16.28	7140.99
scb10x/typhoon-7b	1350022.03	162.00	71053.79
unsloth/mistral-7b-v0.2	1350022.03	162.00	71053.79
upstage/SOLAR-10.7B-v1.0	1350022.03	162.00	71053.79
yanolja/EEVE-Korean-2.8B-v1.0	1350022.03	162.00	71053.79
yanolja/KoSOLAR-10.7B-v0.2	1350022.03	162.00	71053.79

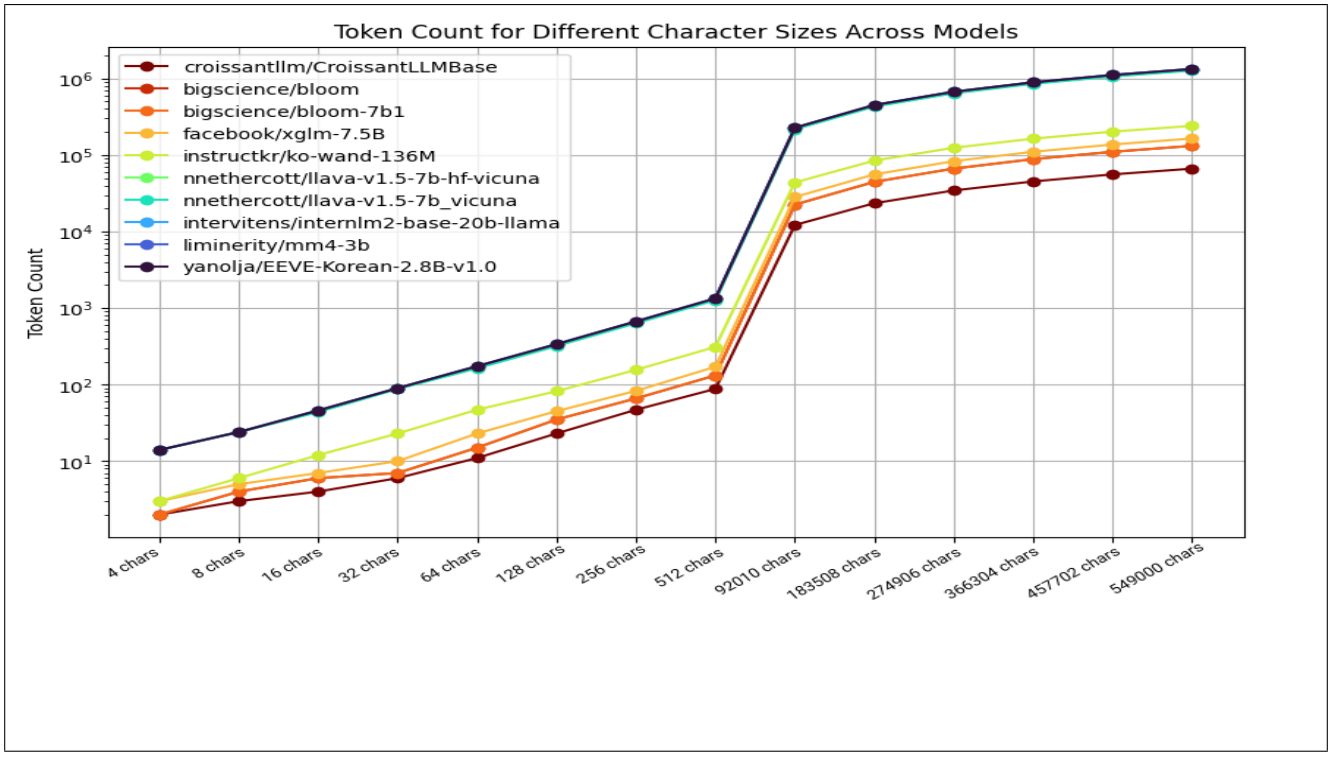


Fig. 28: Token count for different character sizes across models (Gujarati)

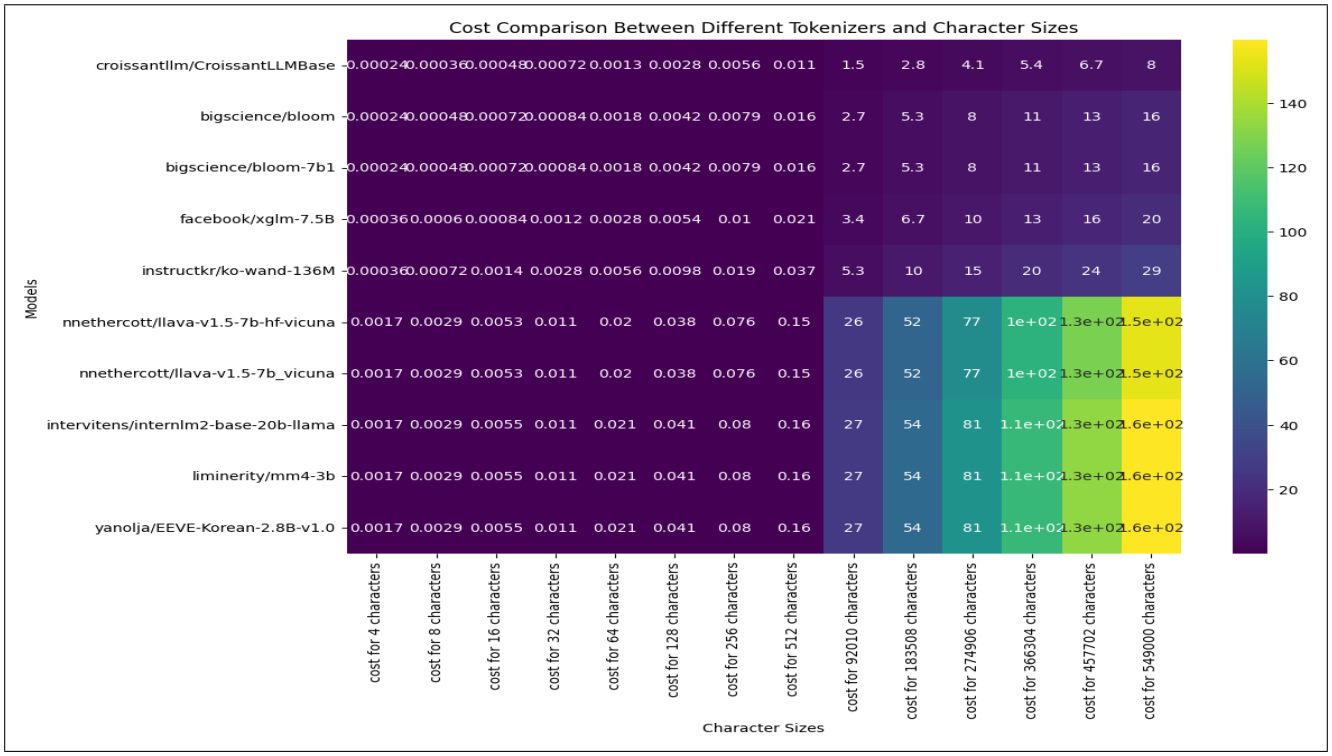


Fig. 29: Cost comparison between different tokenizers and character sizes (Gujarati)

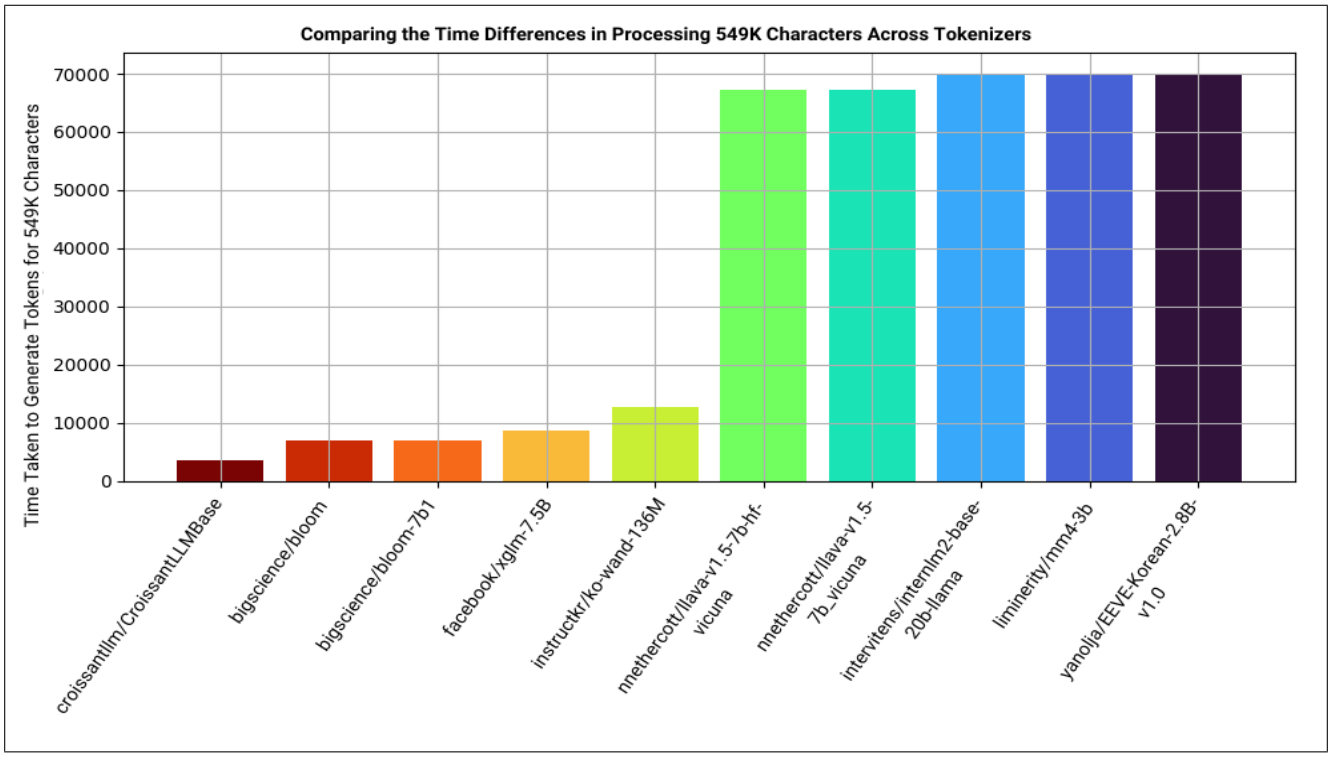


Fig. 30: Comparing the time differences in processing 549K characters across tokenizers (Gujarati)

TABLE X: Tokenizer performance metrics for token sizes across various tokenizers for 549,000 characters in Gujarati language.

Model	Tokens	Cost ($)	Throughput in sec
croissantllm/CroissantLLMBase	66363.34	7.96	3492.81
bigscience/bloom	131922.39	15.83	6943.28
bigscience/bloom-7b1	131922.39	15.83	6943.28
facebook/xglm-7.5B	164286.88	19.71	8646.68
instructkr/ko-wand-136M	240712.91	28.89	12669.10
nnethercott/llava-v1.5-7b-hf-vicuna	1275654.96	153.08	67139.73
nnethercott/llava-v1.5-7b-vicuna	1275654.96	153.08	67139.73
intervitens/internlm2-base-20b-llama	1330978.77	159.72	70051.51
liminerity/mm4-3b	1330978.77	159.72	70051.51
yanolja/EEVE-Korean-2.8B-v1.0	1330978.77	159.72	70051.51

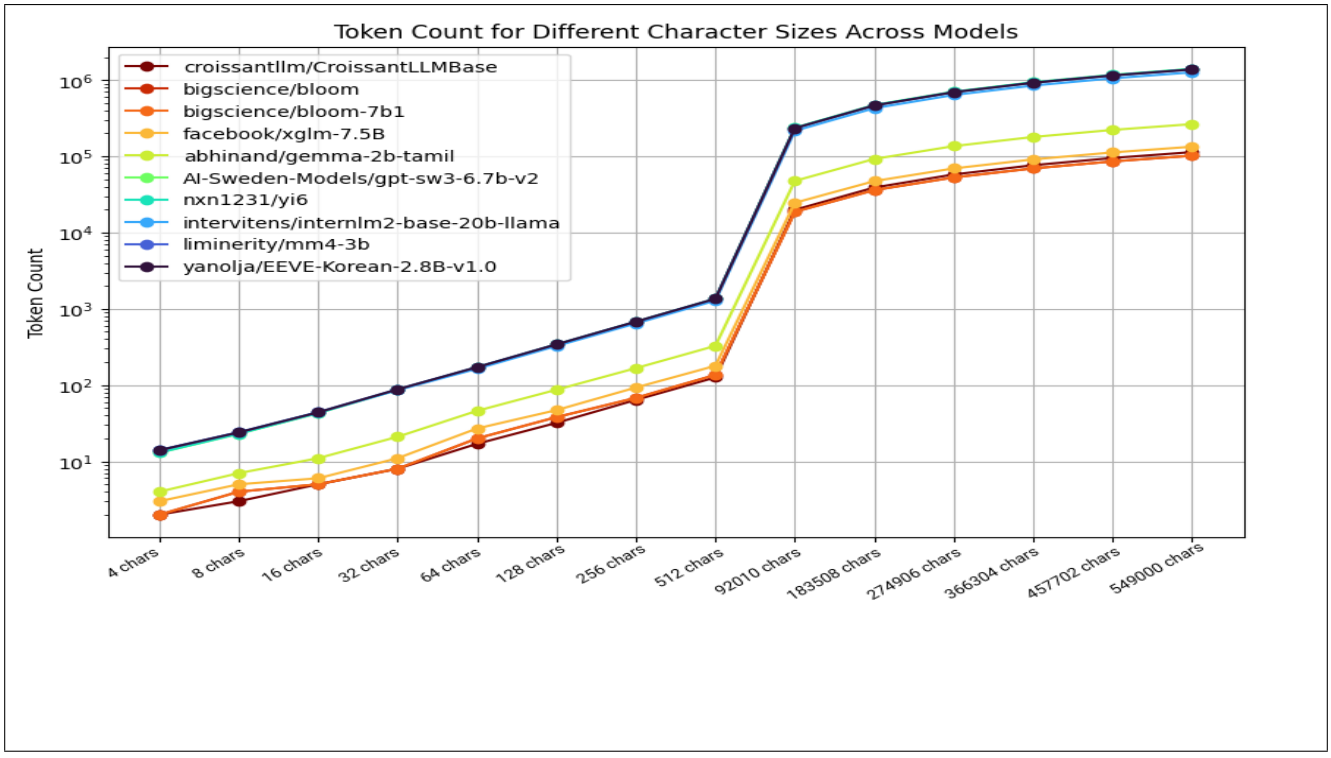


Fig. 31: Token count for different character sizes across models (Assamese)

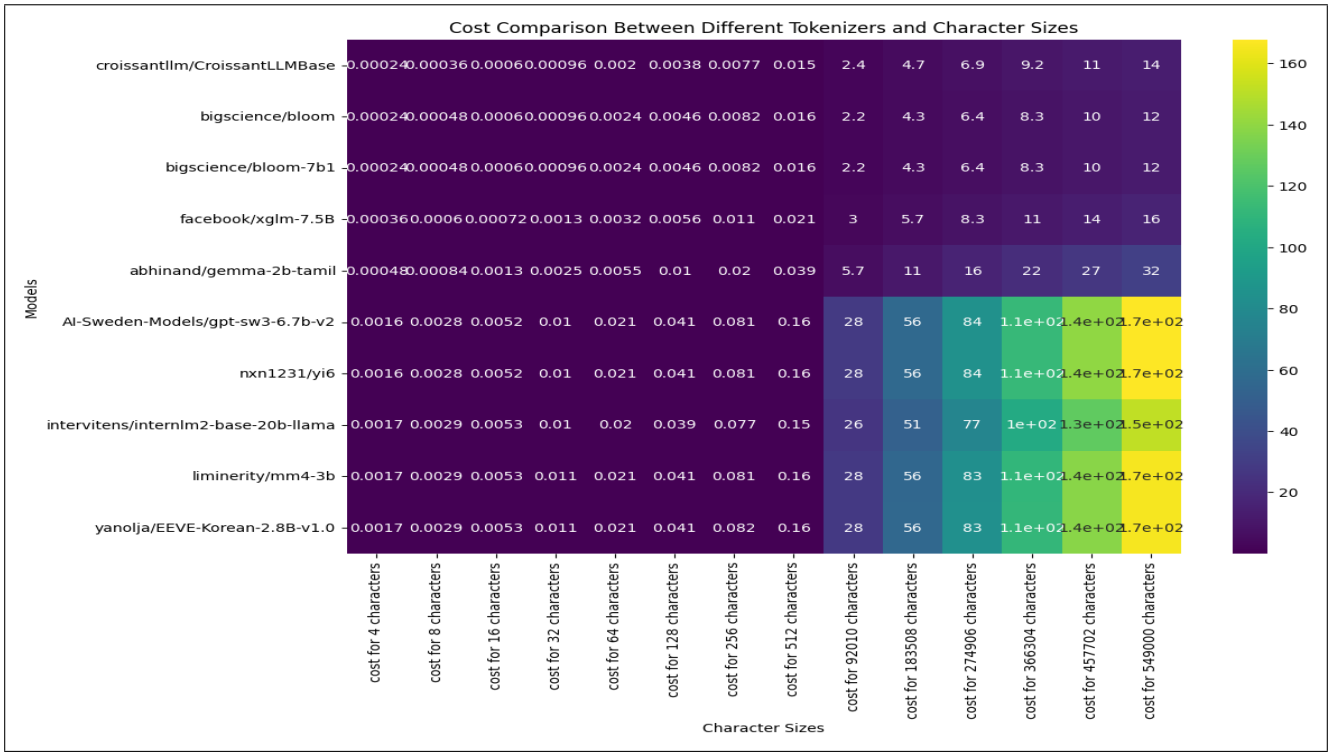


Fig. 32: Cost comparison between different tokenizers and character sizes (Assamese)

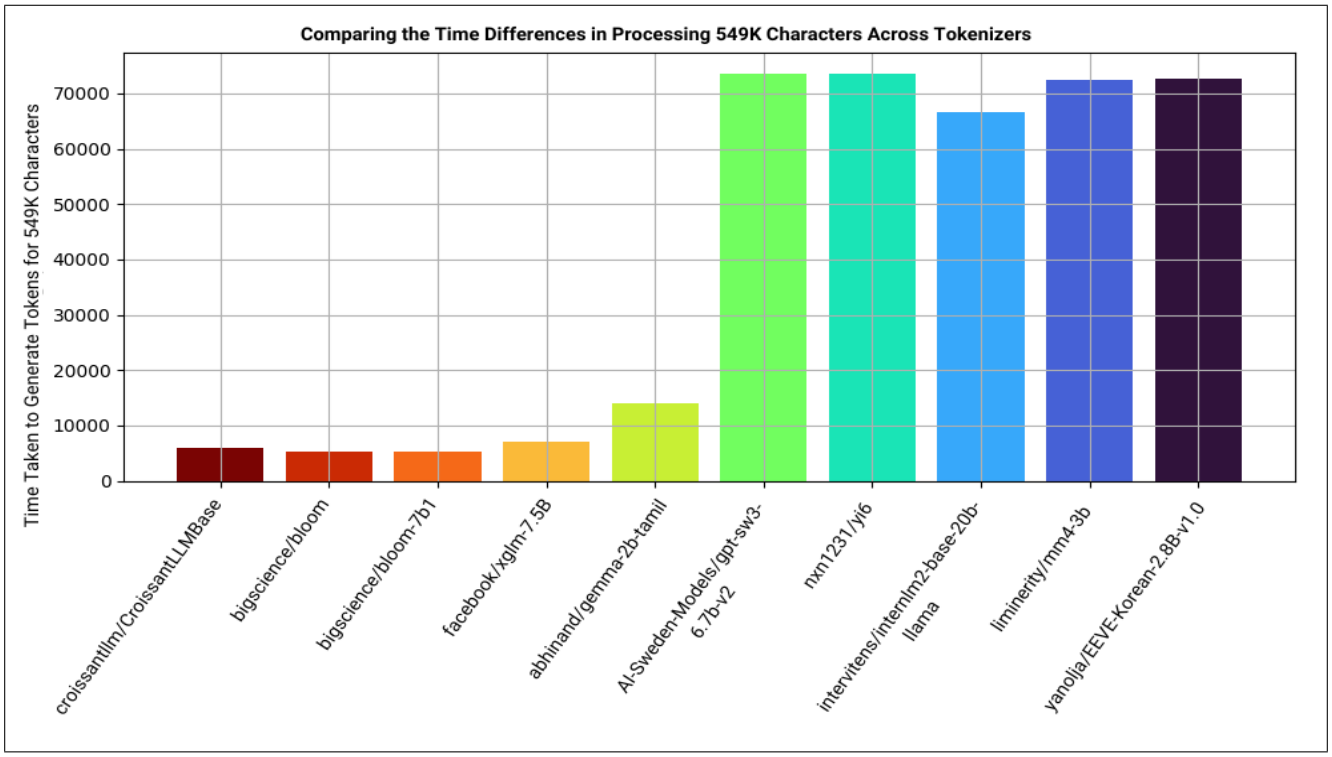


Fig. 33: Comparing the time differences in processing 549K characters across tokenizers (Assamese)

TABLE XI: Tokenizer performance metrics for token sizes across various tokenizers for 549,000 characters in Assamese language.

Model	Tokens	Cost ($)	Throughput in sec
bigscience/bloom	102053.80	12.25	5371.25
bigscience/bloom-7b1	102053.80	12.25	5371.25
croissantllm/CroissantLLMBase	113615.28	13.63	5979.75
facebook/xglm-7.5B	133853.05	16.06	7044.90
abhinand/gemma-2b-tamil	264683.95	31.76	13930.73
intervitens/internlm2-base-20b-llama	1264505.87	151.74	66552.94
liminerity/mm4-3b	1377105.93	165.25	72479.26
yanolja/EEVE-Korean-2.8B-v1.0	1379569.96	165.55	72608.95
AI-Sweden-Models/gpt-sw3-6.7b-v2	1398345.13	167.80	73597.11
nxn1231/yi6	1398345.13	167.80	73597.11

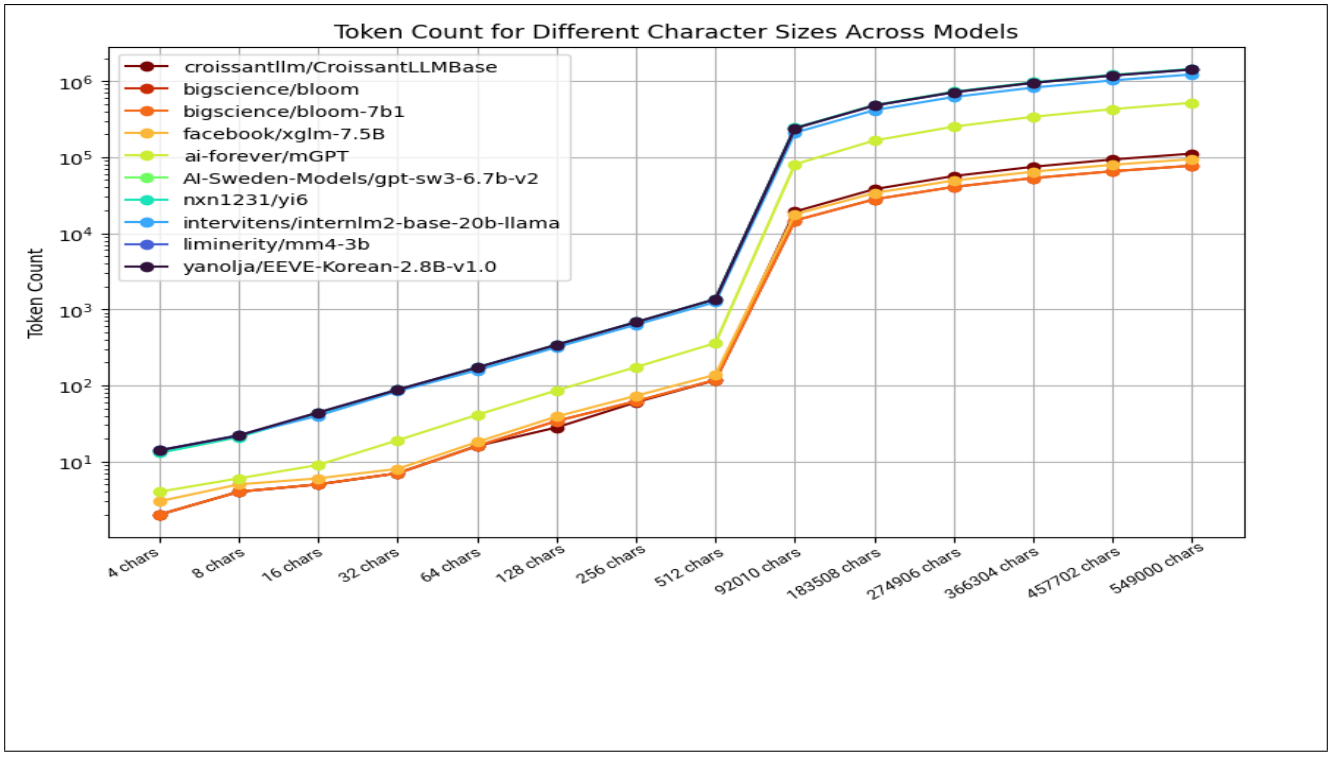


Fig. 34: Token count for different character sizes across models (Bengali)

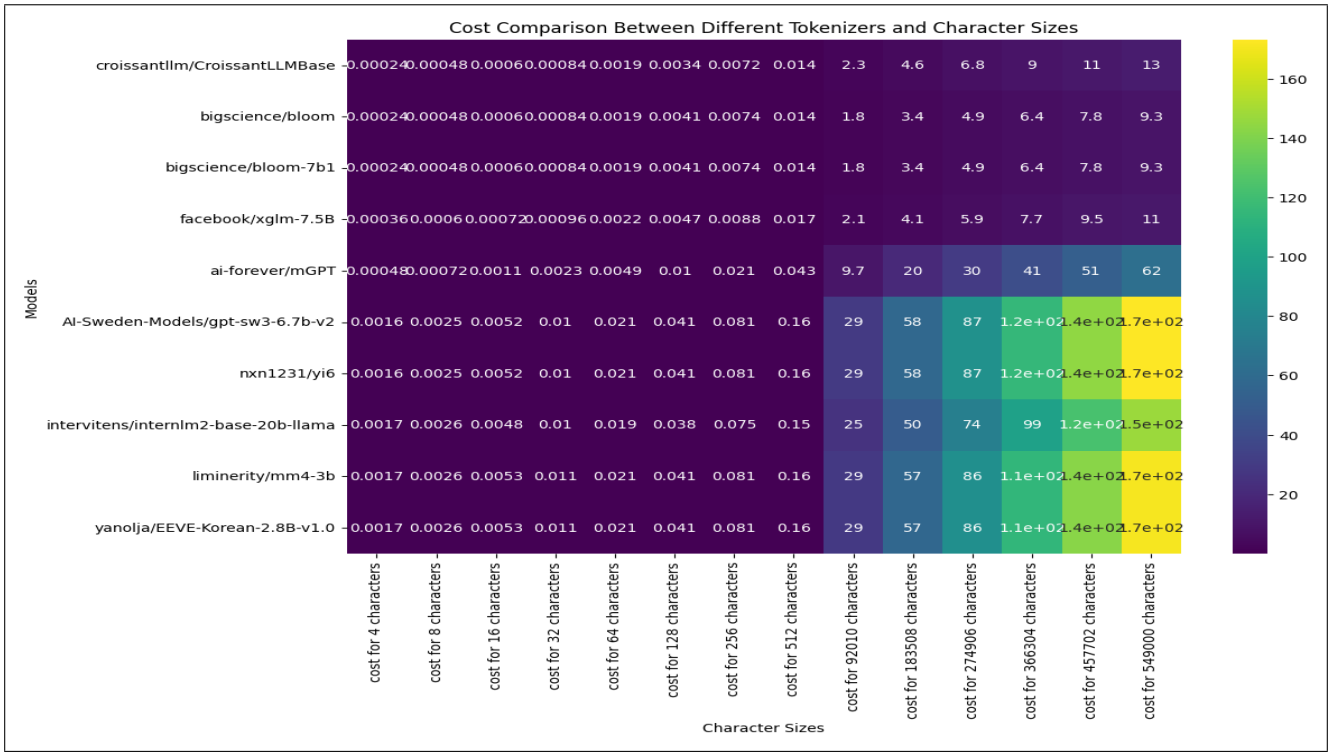


Fig. 35: Cost comparison between different tokenizers and character sizes (Bengali)

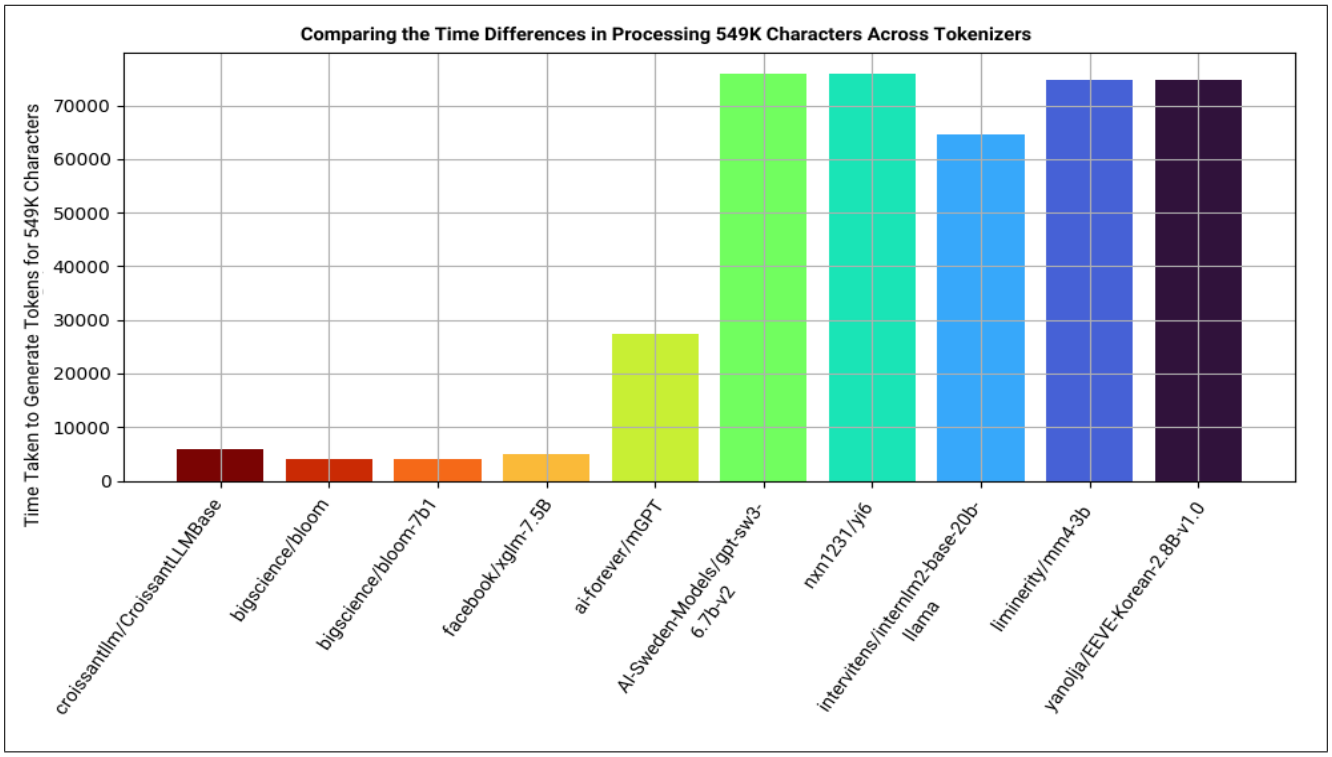


Fig. 36: Comparing the time differences in processing 549K characters across tokenizers (Bengali)

TABLE XII: Tokenizer performance metrics for token sizes across various tokenizers for 549,000 characters in Bengali language.

Model	Tokens	Cost ($)	Throughput in sec
bigscience/bloom	77389.61	9.29	4073.14
bigscience/bloom-7b1	77389.61	9.29	4073.14
facebook/xglm-7.5B	94222.77	11.31	4959.09
croissantllm/CroissantLLMBase	111493.65	13.38	5868.09
ai-forever/mGPT	518721.67	62.25	27301.14
intervitens/internlm2-base-20b-llama	1225430.45	147.05	64496.34
liminerity/mm4-3b	1421266.13	170.55	74803.48
yanolja/EEVE-Korean-2.8B-v1.0	1421266.13	170.55	74803.48
AI-Sweden-Models/gpt-sw3-6.7b-v2	1443213.95	173.19	75958.63
nxn1231/yi6	1443213.95	173.19	75958.63

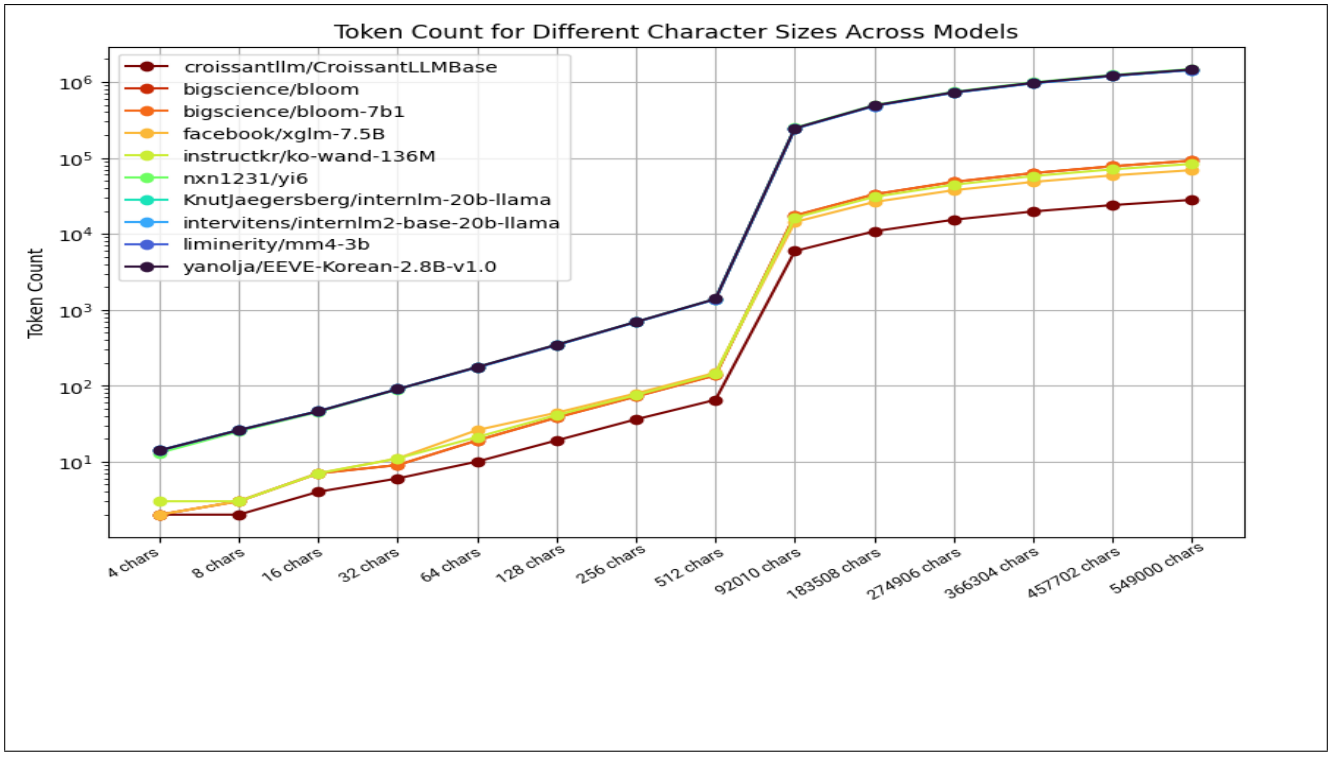


Fig. 37: Token count for different character sizes across models (Telugu)

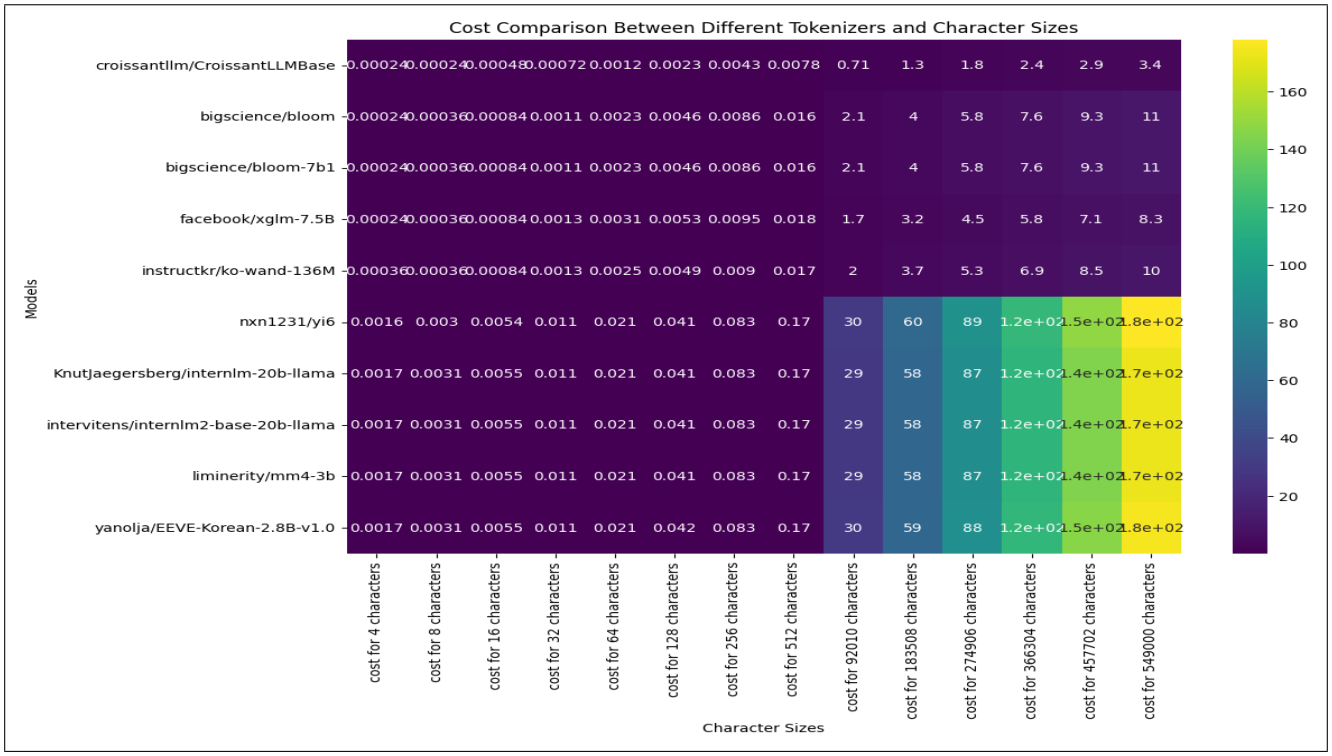


Fig. 38: Cost comparison between different tokenizers and character sizes (Telugu)

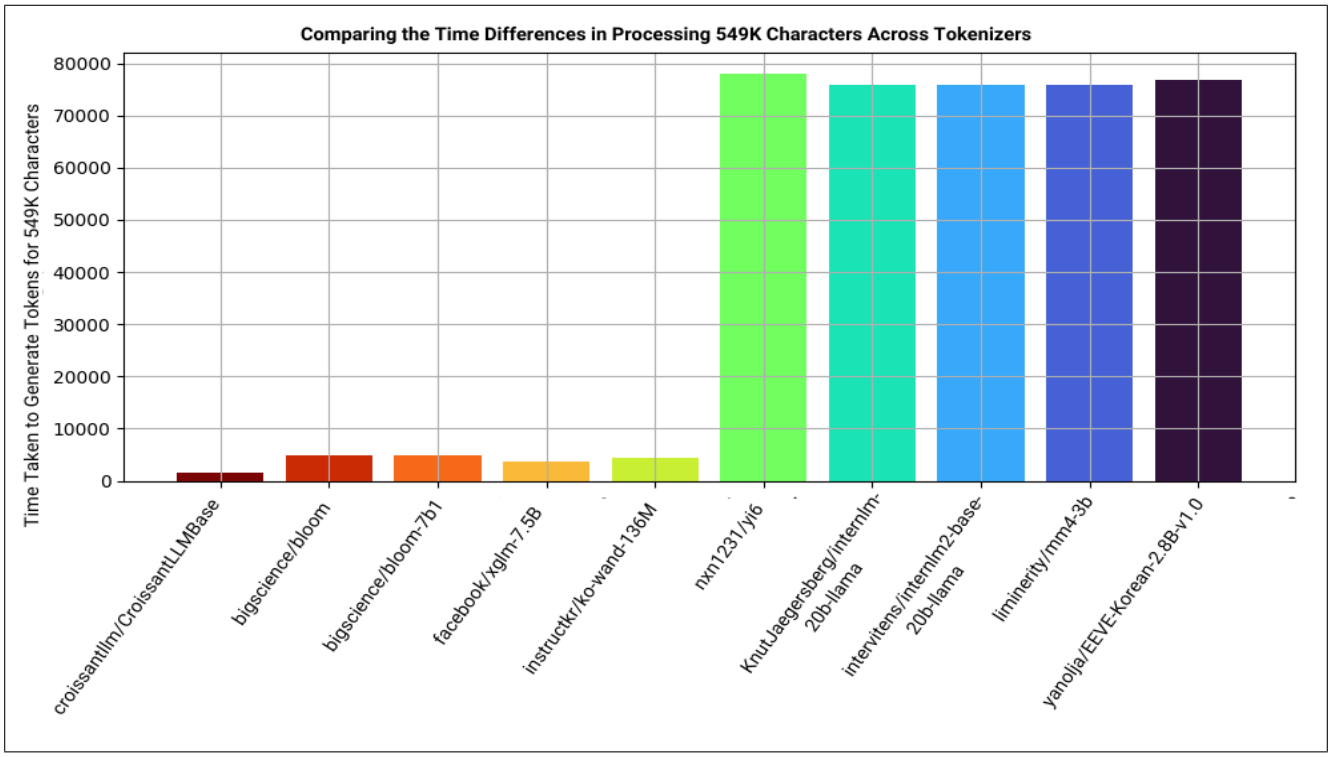


Fig. 39: Comparing the time differences in processing 549K characters across tokenizers (Telugu)

TABLE XIII: Tokenizer performance metrics for token sizes across various tokenizers for 549,000 characters in Telugu language.

Model	Tokens	Cost ($)	Throughput in sec
croissantllm/CroissantLLMBase	28025.71	3.36	1475.04
facebook/xglm-7.5B	69057.24	8.29	3634.59
instructkr/ko-wand-136M	83504.43	10.02	4394.97
bigscience/bloom	92121.11	11.05	4848.48
bigscience/bloom-7b1	92121.11	11.05	4848.48
KnutJaegersberg/internlm-20b-llama	1442664.51	173.12	75929.71
intervitens/internlm2-base-20b-llama	1442664.51	173.12	75929.71
liminerity/mm4-3b	1442664.51	173.12	75929.71
yanolja/EEVE-Korean-2.8B-v1.0	1461062.20	175.33	76898.01
nxn1231/yi6	1483257.78	177.99	78066.20

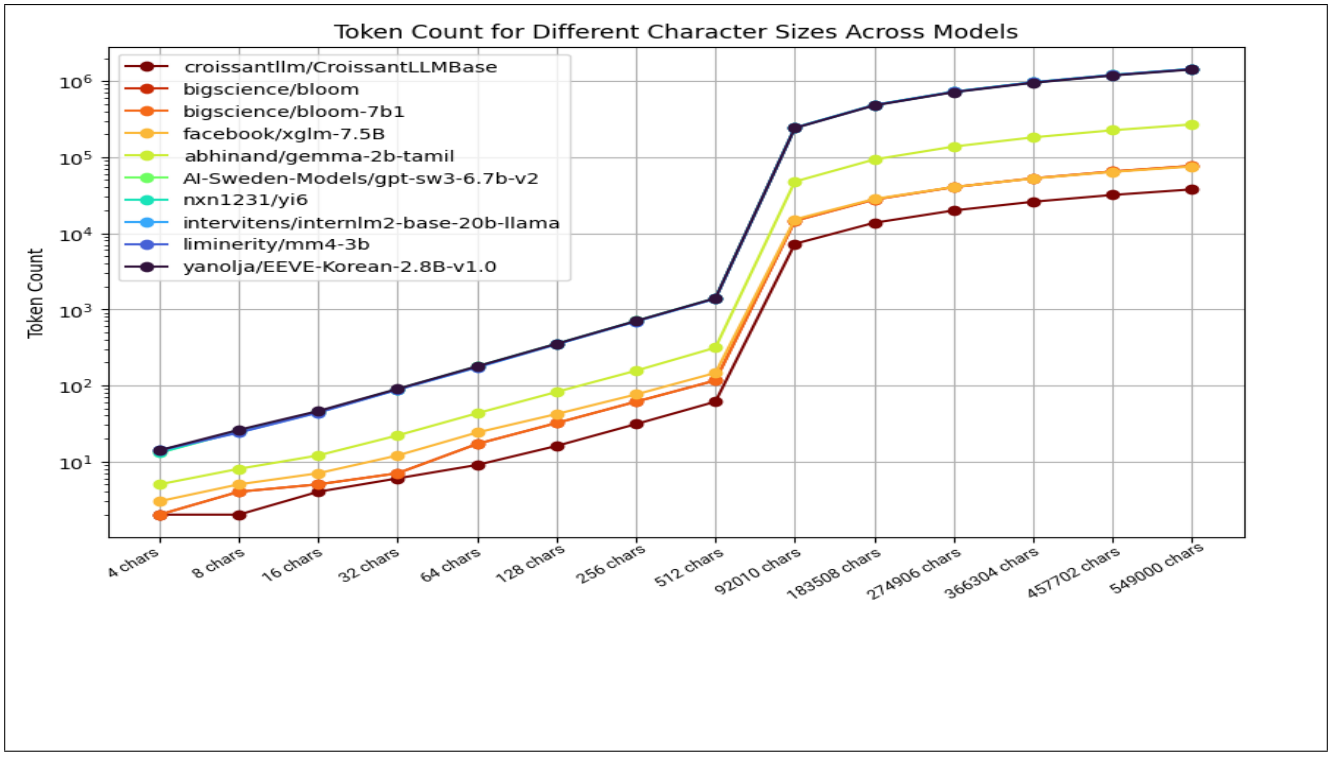


Fig. 40: Token count for different character sizes across models (Kannada)

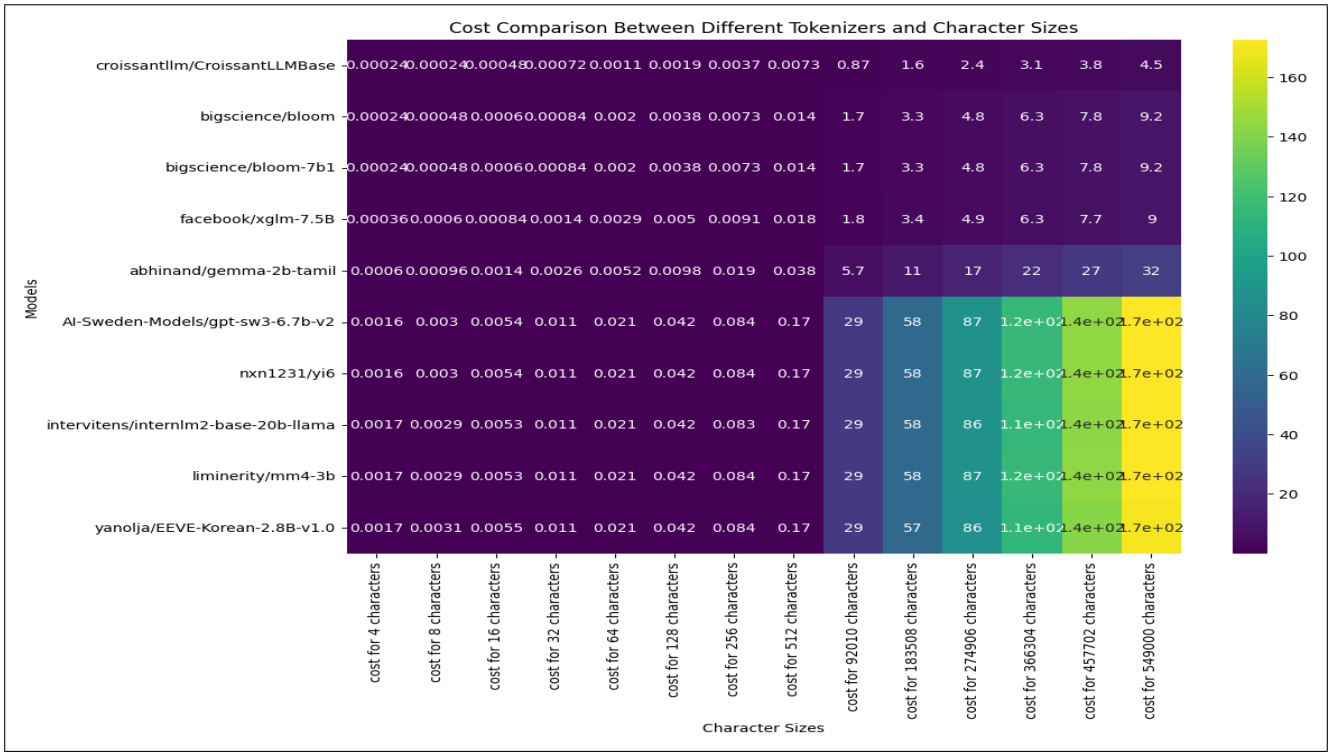


Fig. 41: Cost comparison between different tokenizers and character sizes (Kannada)

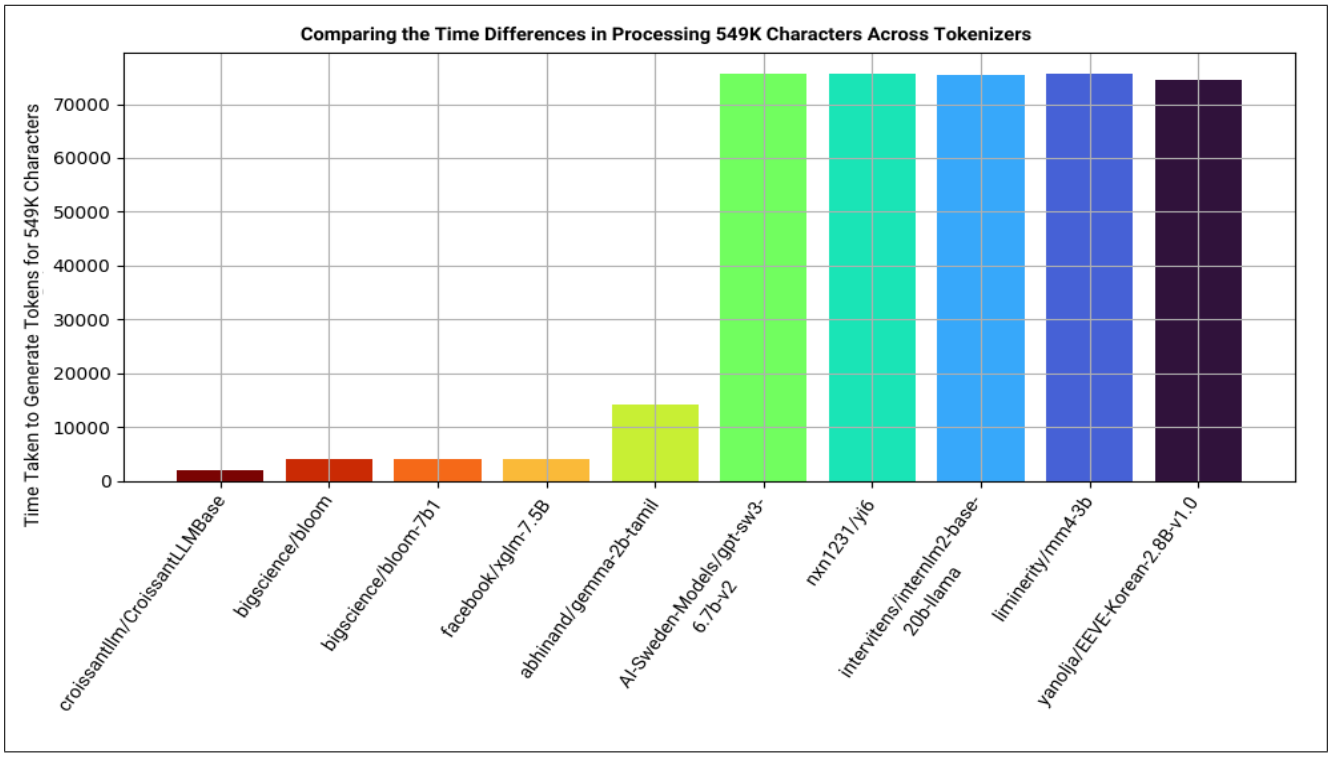


Fig. 42: Comparing the time differences in processing 549K characters across tokenizers (Kannada)

TABLE XIV: Tokenizer performance metrics for token sizes across various tokenizers for 549,000 characters in Kannada language.

Model	Tokens	Cost ($)	Throughput in sec
croissantllm/CroissantLLMBase	37638.17	4.52	1980.96
facebook/xglm-7.5B	75231.31	9.03	3959.54
bigscience/bloom	76747.56	9.21	4039.35
bigscience/bloom-7b1	76747.56	9.21	4039.35
abhinand/gemma-2b-tamil	269068.02	32.29	14161.47
yanolja/EEVE-Korean-2.8B-v1.0	1417197.44	170.06	74589.34
intervitens/internlm2-base-20b-llama	1432968.22	171.96	75419.38
AI-Sweden-Models/gpt-sw3-6.7b-v2	1438324.26	172.60	75701.28
nxn1231/yi6	1438324.26	172.60	75701.28
liminerity/mm4-3b	1438560.39	172.63	75713.70

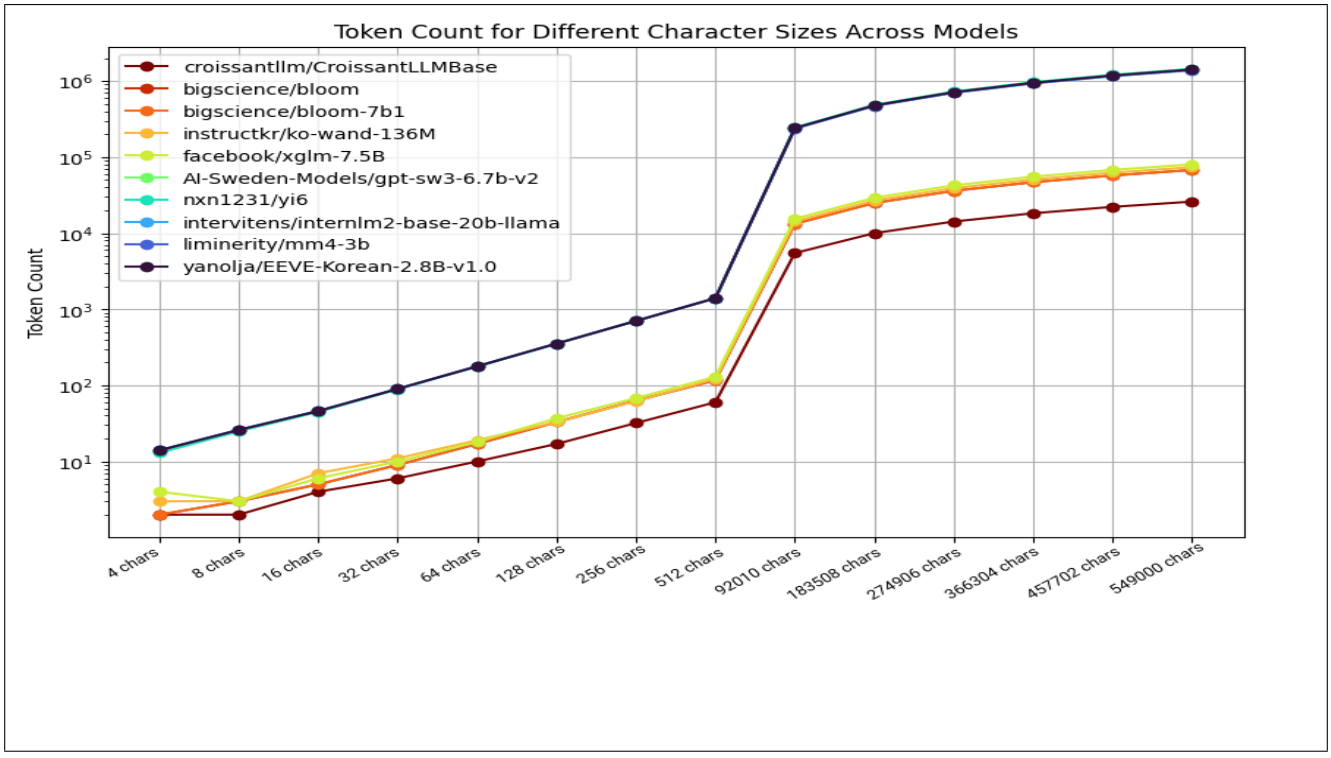


Fig. 43: Token count for different character sizes across models (Malayalam)

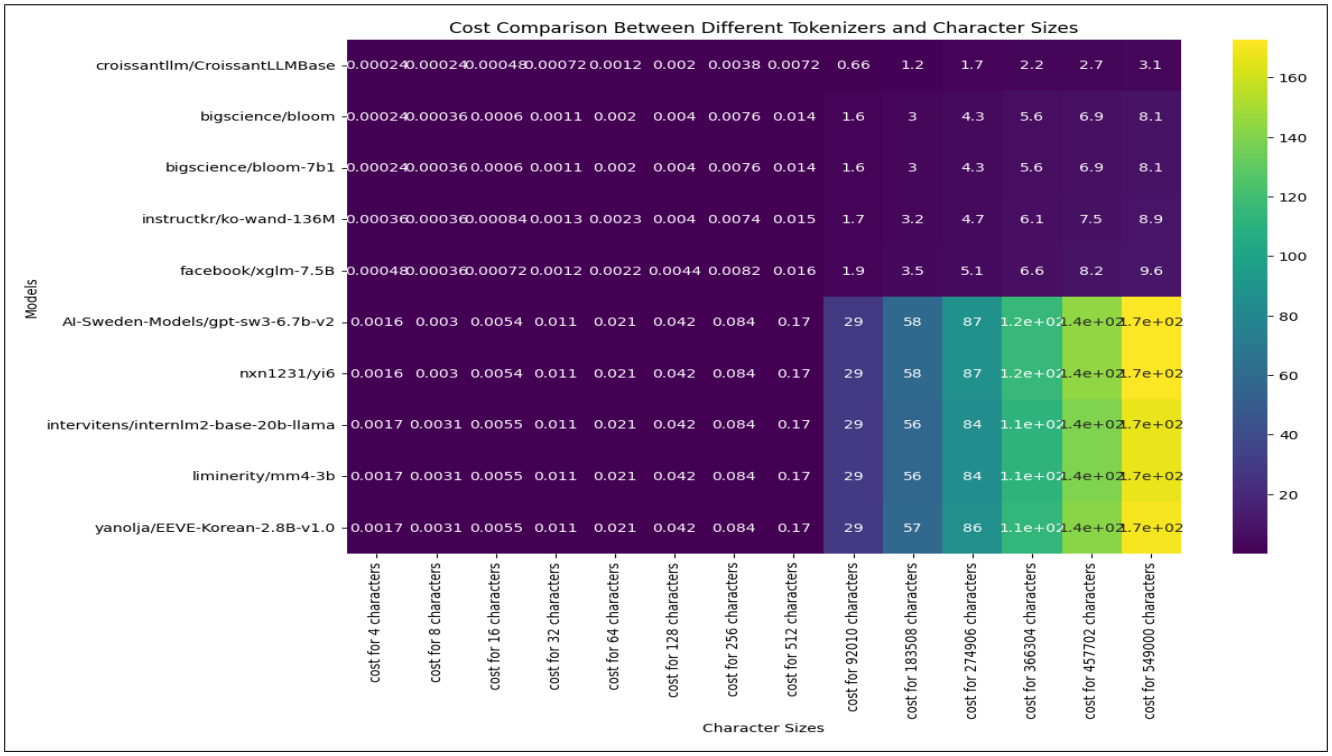


Fig. 44: Cost comparison between different tokenizers and character sizes (Malayalam)

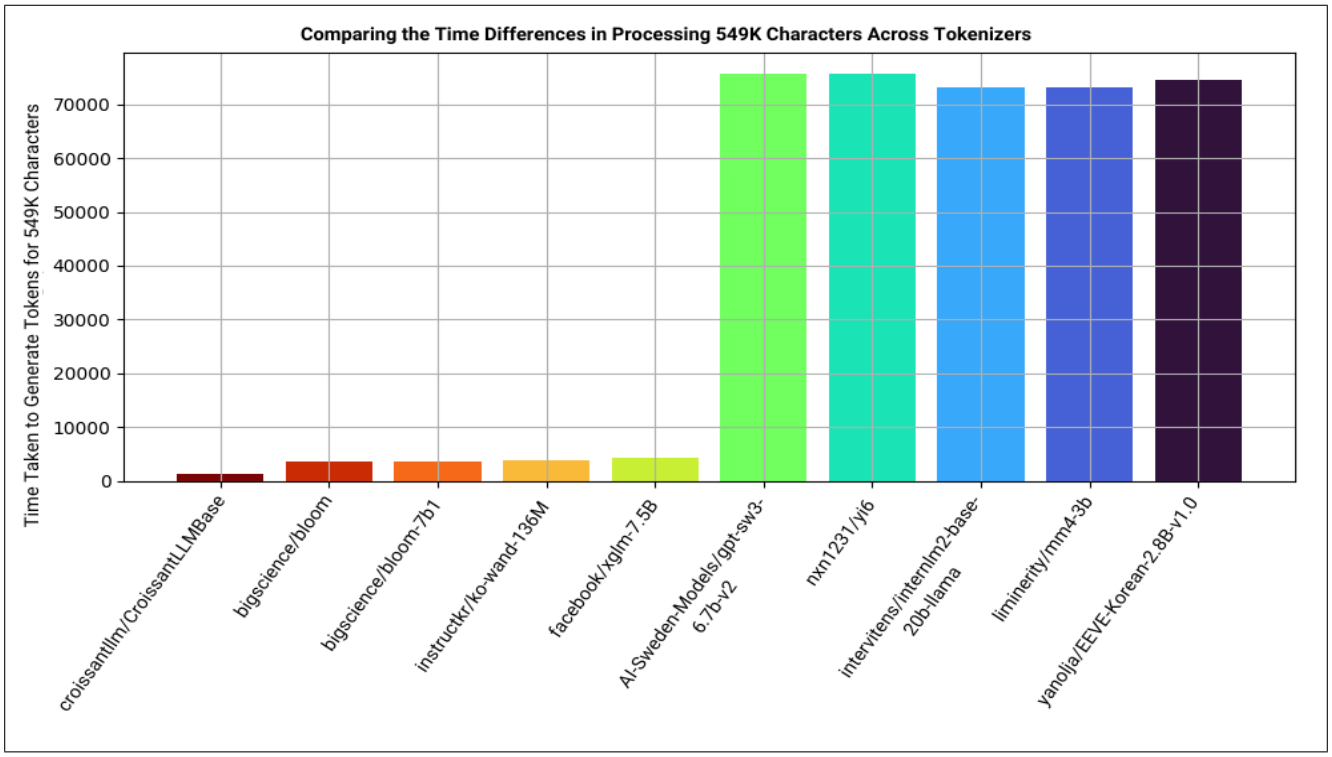


Fig. 45: Comparing the time differences in processing 549K characters across tokenizers (Malayalam)

TABLE XV: Tokenizer performance metrics for token sizes across various tokenizers for 549,000 characters in Malayalam language.

Model	Tokens	Cost ($)	Throughput in sec
croissantllm/CroissantLLMBase	26006.11	3.12	1368.74
bigscience/bloom	67794.78	8.14	3568.15
bigscience/bloom-7b1	67794.78	8.14	3568.15
instructkr/ko-wand-136M	73860.88	8.86	3887.41
facebook/xglm-7.5B	80383.41	9.65	4230.71
intervitens/internlm2-base-20b-llama	1392277.27	167.07	73277.75
liminerity/mm4-3b	1392277.27	167.07	73277.75
yanolja/EEVE-Korean-2.8B-v1.0	1418696.60	170.24	74668.24
AI-Sweden-Models/gpt-sw3-6.7b-v2	1439765.45	172.77	75777.13
nxn1231/yi6	1439765.45	172.77	75777.13